Introduction and Setup

1 card

1. Introduction

INTRODUCTION

Scenarios

4 cards

1. Software Supply Chain Trace Lab
2. Payara + mvnpm Supply Chain Trace Lab
3. Maven Plugin Hidden-Content Supply Chain Trace Lab
4. S05 — Node npm prepack Supply Chain Trace Lab

SOFTWARE SUPPLY CHAIN TRACE LAB

This lab follows three tracer components through a small but realistic software supply chain:

- `jackson-databind` — an ordinary Java dependency whose version is selected by dependency management.
- `commons-codec` — a Java dependency whose bytecode is shaded and relocated into another JAR.
- `lodash` — an npm dependency that is bundled into frontend JavaScript and loses its package boundary.

The point is not to teach Maven, npm, Docker, or SBOM basics. The point is to see **what evidence exists at each stage, what survives the next transformation, and what different tools can legitimately know.**

The pattern throughout is:

Look → Run → Observe → Establish

The exercise ends at the container-image boundary. Reverse provenance from a binary/container back to a source repository and exact Git commit is deliberately left for a separate exercise.

WHY WE NEED TO DO THIS

Supply-chain tracing is only useful if we know which build produced the artefacts we are inspecting. Old `target/`, `dist/`, SBOM, or experiment output can make later evidence ambiguous.

We therefore begin by removing generated state while keeping source and lockfiles intact.

HOW WE'RE GOING TO DO IT

The project contains a `clean.sh` wrapper so every run starts from the same state. It removes Maven targets, npm-installed modules, frontend build output, SBOM output, and controlled-experiment output. It deliberately preserves `package-lock.json` because that is part of the dependency-resolution evidence.

RUN

```
./scripts/clean.sh
```

ESTABLISH

Generated build, frontend, trace, and SBOM artefacts are removed. Source and lockfiles remain.

WHY WE NEED TO DO THIS

Before asking a resolver or scanner what exists, we need to know what the developers actually asked the build to do.

A declaration is only intent. It may contain no version, may inherit a version from elsewhere, may later be overridden, or may describe a transformation that never actually happens. We will keep that distinction visible throughout the lab.

HOW WE'RE GOING TO DO IT

Look at the source configuration directly rather than asking Maven or npm to reinterpret it for us. This gives us the literal declarations against which later resolver and artefact evidence can be compared.

Look at:

```
service/pom.xml
normalizer/pom.xml
pom.xml
frontend/package.json
```

JACKSON

`service/pom.xml` declares `jackson-databind` without a local version. The root `pom.xml` imports the Spring Boot dependency-management BOM.

```
jackson-databind
    declared by service
    version not specified locally
```

At this point we know Jackson is requested, but not which version Maven will select.

COMMONS-CODEC

`normalizer/pom.xml` declares `commons-codec` using the property `${commons-codec.version}`. The root `pom.xml` defines that property as `1.17.1`.

The Maven Shade Plugin is also configured to relocate:

```
org.apache.commons.codec
        ↓
com.acme.internal.codec
```

At source-inspection time this is only the intended transformation.

LODASH

`frontend/package.json` declares:

```
lodash: 4.17.21
```

ESTABLISH

These are source declarations/configuration, not proof of what gets resolved or shipped.

WHY WE NEED TO DO THIS

The interesting evidence only appears once the build has actually run. Resolution chooses concrete versions; Vite changes JavaScript package structure; Shade rewrites bytecode; Spring Boot assembles the executable JAR.

Until those transformations happen, we only have configuration.

HOW WE'RE GOING TO DO IT

`build.sh` runs the project's normal build path in the correct order rather than reproducing it manually in the lab. The important thing is that we inspect the outputs produced by the same build path the application actually uses.

RUN

```
./scripts/build.sh
```

ESTABLISH

The build performs:

```
npm
    resolves and installs dependencies

Vite
    creates the frontend bundle

Maven
    resolves Java dependencies

Maven Shade
    builds the transformed normalizer

Spring Boot
    builds the executable service JAR
```

WHY WE NEED TO DO THIS

The source POM did not state a Jackson version. The version that matters to the build is the one Maven resolves after applying dependency management.

This is our first example of the difference between **declared dependency** and **resolved dependency**.

HOW WE'RE GOING TO DO IT

We ask Maven's dependency plugin for the resolved dependency tree of the `service` module.

`-pl service` selects the `service` module from the reactor.

`-am` means "also make" the projects that `service` depends on, so reactor dependencies are available rather than being treated as missing external artefacts.

`-Dincludes=...` filters the tree to the component we are tracing; it does not affect resolution.

RUN

```
mvn -pl service -am dependency:tree \
  -Dincludes=com.fasterxml.jackson.core:jackson-databind
```

OBSERVED OUTPUT

```
[INFO] dev.noregressions.trace:service:jar:1.0.0
[INFO] \- com.fasterxml.jackson.core:jackson-databind:jar:2.19.4:compile
```

ESTABLISH

```
Source
    jackson-databind
    no local version

        ↓ Maven resolution

Resolved
    jackson-databind:2.19.4
```

This proves Maven selected `jackson-databind:2.19.4`. It does not yet prove it was packaged.

WHY WE NEED TO DO THIS

`commons-codec` is going to be transformed by Shade. Before looking at that transformation, we need a baseline: which version did the `normalizer` module actually resolve before shading?

Without that baseline we could not later tell whether the shaded code and metadata still correspond to the resolved component.

HOW WE'RE GOING TO DO IT

Again we use Maven's resolved dependency tree, but this time for `normalizer`. The `-Dincludes` filter keeps the result focused on `commons-codec`.

RUN

```
mvn -pl normalizer dependency:tree \
  -Dincludes=commons-codec:commons-codec
```

OBSERVED OUTPUT

```
[INFO] dev.noregressions.trace:normalizer:jar:1.0.0
[INFO] \- commons-codec:commons-codec:jar:1.17.1:compile
```

ESTABLISH

```
Source configuration
    commons-codec version property → 1.17.1

        ↓ Maven resolution

Resolved
    commons-codec:1.17.1
```

WHY WE NEED TO DO THIS

Frontend dependencies behave differently from Java JARs. Lodash will later be folded into a generated JavaScript bundle, so now is the point where its package identity is strongest and easiest to prove.

We want both the installed package view and the lockfile evidence before bundling destroys that structure.

HOW WE'RE GOING TO DO IT

`npm ls lodash` asks npm to show the installed dependency graph for just lodash.

The lockfile gives us stronger input evidence: exact version, registry tarball, and integrity hash. We inspect the lockfile directly rather than asking another tool to infer it.

From `frontend/`:

RUN

```
npm ls lodash
```

OBSERVED OUTPUT

```
checkout-trace-frontend@1.0.0
└── lodash@4.17.21
```

Look at the `node_modules/lodash` entry in `frontend/package-lock.json`.

OBSERVED LOCKFILE ENTRY

```
"node_modules/lodash": {
  "version": "4.17.21",
  "resolved": "https://registry.npmjs.org/lodash/-/lodash-4.17.21.tgz",
  "integrity": "sha512-v2kDEe57lecTulaDIuNTPy3Ry4gLGJ6Z1O3vE1krgXZNrsQ+LFTGHVx
  "license": "MIT"
}
```

ESTABLISH

```
jackson-databind  2.19.4
commons-codec      1.17.1
lodash             4.17.21
```

These are our resolved tracer versions before packaging transformations obscure or duplicate them.

WHY WE NEED TO DO THIS

A dependency tree tells us what Maven resolved. It does not prove that those bytes ended up in the product.

Jackson is our uncomplicated control case: if Spring Boot packages it normally, we should find an ordinary nested JAR with the same version Maven resolved.

HOW WE'RE GOING TO DO IT

Spring Boot executable JARs store dependency JARs under `BOOT-INF/lib`. We list the archive and filter for `jackson-databind`.

`unzip -l` lists archive entries without extracting them. The `grep` is only narrowing a generated artefact listing; it is not being used to inspect source.

RUN

```
unzip -l service/target/service-1.0.0.jar \
  | grep jackson-databind
```

OBSERVED OUTPUT

```
1679441  10-30-2025 00:03   BOOT-INF/lib/jackson-databind-2.19.4.jar
```

ESTABLISH

```
Maven resolution
    jackson-databind:2.19.4

        ↓ Spring Boot packaging

service-1.0.0.jar
    BOOT-INF/lib/jackson-databind-2.19.4.jar
```

Jackson `2.19.4` is both resolved and physically packaged.

WHY WE NEED TO DO THIS

The Shade configuration said commons-codec would be relocated. We need to prove the transformation actually happened and see what it did to the component's visible structure.

This is where a simple dependency graph starts to diverge from the physical artefact.

HOW WE'RE GOING TO DO IT

We inspect the built normalizer JAR, not the POM. First we look for classes under the new namespace, then verify that the old namespace is absent.

`head` keeps the listing small; the important evidence is the path prefix, not every class name.

RUN

```
unzip -l normalizer/target/normalizer-1.0.0.jar \
  | grep 'com/acme/internal/codec' \
  | head
```

OBSERVED OUTPUT

```
        0  07-12-2024 16:14   com/acme/internal/codec/
      265  07-12-2024 16:14   com/acme/internal/codec/BinaryDecoder.class
      265  07-12-2024 16:14   com/acme/internal/codec/BinaryEncoder.class
      911  07-12-2024 16:14   com/acme/internal/codec/CharEncoding.class
     1306  07-12-2024 16:14   com/acme/internal/codec/Charsets.class
     1091  07-12-2024 16:14   com/acme/internal/codec/CodecPolicy.class
```

Now check for the original namespace.

RUN

```
unzip -l normalizer/target/normalizer-1.0.0.jar \
  | grep 'org/apache/commons/codec' \
  | head
```

OBSERVED OUTPUT

```
(no output)
```

ESTABLISH

```
org.apache.commons.codec.*
        ↓ Shade
com.acme.internal.codec.*
```

The configured relocation actually occurred.

WHY WE NEED TO DO THIS

Relocating class names does not necessarily erase all evidence of the original component. Many Java archives carry Maven metadata under `META-INF/maven`.

Whether that metadata survives matters because downstream inventory tools may use it to recognise software even when the bytecode namespace has changed completely.

HOW WE'RE GOING TO DO IT

First we list the commons-codec Maven metadata entries inside the shaded JAR. Then `unzip -p` streams the contents of `pom.properties` to stdout without extracting a file to disk.

RUN

```
unzip -l normalizer/target/normalizer-1.0.0.jar \
  | grep 'META-INF/maven/commons-codec'
```

OBSERVED OUTPUT

```
META-INF/maven/commons-codec/
META-INF/maven/commons-codec/commons-codec/
```

```
META-INF/maven/commons-codec/commons-codec/pom.xml
META-INF/maven/commons-codec/commons-codec/pom.properties
```

Inspect the identity metadata.

RUN

```
unzip -p normalizer/target/normalizer-1.0.0.jar \
  META-INF/maven/commons-codec/commons-codec/pom.properties
```

OBSERVED OUTPUT

```
artifactId=commons-codec
groupId=commons-codec
version=1.17.1
```

ESTABLISH

The Java namespace changed, but the original Maven component identity survived.

WHY WE NEED TO DO THIS

So far we have manually correlated relocated classes with surviving Maven metadata. A real SBOM or inventory workflow needs a tool to make that identification from the artefact itself.

This step asks whether an independent artefact scanner can recover `commons-codec:1.17.1` from the finished shaded JAR.

HOW WE'RE GOING TO DO IT

Syft is a software package cataloguer from Anchore. Given a filesystem, archive, or container image, it looks for package-identifying evidence such as package-manager metadata and archive metadata and emits a software inventory.

We are not giving Syft the Maven dependency tree or the project source here. Its input is only `normalizer-1.0.0.jar`.

RUN

```
syft normalizer/target/normalizer-1.0.0.jar
```

OBSERVED OUTPUT

```
NAME           VERSION  TYPE
commons-codec  1.17.1   java-archive
normalizer     1.0.0    java-archive
```

ESTABLISH

```
relocated code
    +
```

```
surviving Maven metadata
    ↓
commons-codec 1.17.1 identified
```

The transformed class names did not prevent component identification because identifying package evidence survived in the artefact.

WHY WE NEED TO DO THIS

The previous result is correlation: the metadata exists and Syft finds the package. To understand whether that evidence is actually important, we need a controlled change.

We will remove only the identifying Maven metadata while leaving the relocated commons-codec bytecode unchanged, then repeat the same scan. If the result changes, the difference is attributable to the evidence we removed.

HOW WE'RE GOING TO DO IT

`strip-codec-metadata.sh` creates a copy of the shaded normalizer JAR, removes only `META-INF/maven/commons-codec/...`, and runs the same Syft check before and after.

The script is intentionally part of the project because the experiment should be repeatable rather than dependent on a one-off sequence of archive-editing commands.

RUN

```
./scripts/strip-codec-metadata.sh
```

OBSERVED OUTPUT

Original Syft result:

```
commons-codec  1.17.1   java-archive
normalizer     1.0.0    java-archive
```

Metadata-stripped result:

```
normalizer                   1.0.0    java-archive
normalizer-no-codec-metadata  UNKNOWN  java-archive
```

ESTABLISH

`commons-codec:1.17.1` disappeared from Syft's inventory although the relocated bytecode remained.

```
Same relocated bytecode
        +
Maven package metadata
        ↓
commons-codec 1.17.1 identified

Same relocated bytecode
        -
Maven package metadata
```

```
        ↓
commons-codec not identified
```

The important distinction is:

```
software presence != software identifiability
```

WHY WE NEED TO DO THIS

Lodash gives us a different transformation. Rather than relocating classes inside an archive, Vite incorporates npm package code into generated frontend assets.

We need to see what remains of the npm package structure after bundling before asking a scanner to identify anything.

HOW WE'RE GOING TO DO IT

We list only the generated files in `frontend/dist`. `find -maxdepth 2` keeps the listing to the deployable output and its immediate asset directories.

RUN

```
find frontend/dist -maxdepth 2 -type f -print
```

OBSERVED OUTPUT

```
frontend/dist/index.html
frontend/dist/assets/index-QMB-eT_H.js
frontend/dist/assets/index-DVImxnjI.css
frontend/dist/.vite/manifest.json
```

ESTABLISH

The discrete npm package structure is no longer present in the deployable frontend output.

We no longer have a deployable `node_modules/lodash/` directory; we have generated JavaScript and CSS assets.

WHY WE NEED TO DO THIS

We know lodash was a build input. The question is whether its package identity survives bundling strongly enough for an artefact scanner to recover `lodash@4.17.21` from the deployable frontend alone.

This is the frontend counterpart to the shaded-Java experiment.

HOW WE'RE GOING TO DO IT

We give Syft only `frontend/dist`. It does not receive `package.json`, `package-lock.json`, or `node_modules` from the build workspace.

That distinction matters: this is an **artefact inspection** question, not a build-input inventory question.

RUN

```
syft frontend/dist
```

OBSERVED OUTPUT

```
✔ Packages [0 packages]
✔ Executables [0 executables]

No packages discovered
```

ESTABLISH

```
npm build input
    lodash@4.17.21

        ↓ Vite bundling

frontend/dist
    generated JavaScript bundle

        ↓ Syft

0 packages identified
```

This does **not** prove lodash-derived code is absent. It proves that this built frontend does not retain enough npm package identity for this Syft scan to identify `lodash@4.17.21`.

Compare this with the shaded Java example:

```
commons-codec
    transformed
    metadata survives
    → component remains identifiable

lodash
    bundled
    package identity evidence disappears
    → component is not identified
```

WHY WE NEED TO DO THIS

The fact that Vite produced a bundle does not prove that bundle reached the application we ship. We need to cross the next packaging boundary.

This keeps the evidence chain continuous: build input → transformed frontend → packaged application.

HOW WE'RE GOING TO DO IT

Spring Boot serves static application resources from its packaged classes area. We inspect the finished executable JAR for the exact files we saw in `frontend/dist`.

RUN

```
unzip -l service/target/service-1.0.0.jar \
  | grep 'BOOT-INF/classes/static/'
```

OBSERVED OUTPUT

```
        0  08-21-2026 13:02   BOOT-INF/classes/static/
        0  08-21-2026 13:02   BOOT-INF/classes/static/assets/
        0  08-21-2026 13:02   BOOT-INF/classes/static/.vite/
      416  08-21-2026 13:02   BOOT-INF/classes/static/index.html
   216542  08-21-2026 13:02   BOOT-INF/classes/static/assets/index-QMB-eT_H.js
      254  08-21-2026 13:02   BOOT-INF/classes/static/assets/index-DVImxnjI.cs
      185  08-21-2026 13:02   BOOT-INF/classes/static/.vite/manifest.json
```

ESTABLISH

The transformed frontend reached the final application.

We can prove the bundle was shipped even though the lodash package identity is no longer recoverable from it.

WHY WE NEED TO DO THIS

We have proved what is inside the standalone normalizer JAR. Now we need to prove that this transformed component reached the final Spring Boot application as well.

HOW WE'RE GOING TO DO IT

Spring Boot packages ordinary dependency JARs under `BOOT-INF/lib`. We inspect the service archive for `normalizer-1.0.0.jar`.

RUN

```
unzip -l service/target/service-1.0.0.jar \
  | grep 'normalizer-1.0.0.jar'
```

OBSERVED OUTPUT

```
373350  08-21-2026 13:02   BOOT-INF/lib/normalizer-1.0.0.jar
```

ESTABLISH

The shaded normalizer is present as a nested JAR in the executable application.

WHY WE NEED TO DO THIS

We have inspected individual tracers manually. Now we want an independent inventory of the **finished application as a whole**.

This is the first point where unexpected software can appear because the scanner sees nested artefacts and surviving metadata that may not be obvious from the top-level dependency model.

HOW WE'RE GOING TO DO IT

We give Syft the complete Spring Boot JAR. Syft recursively catalogs package evidence inside the archive, including nested Java archives.

Its answer is therefore an artefact-derived inventory, not a Maven dependency tree.

RUN

```
syft service/target/service-1.0.0.jar
```

OBSERVED OUTPUT

Syft discovered 34 Java packages. Relevant entries:

```
commons-codec     1.17.1   java-archive
commons-codec     1.18.0   java-archive
jackson-databind  2.19.4   java-archive
normalizer        1.0.0    java-archive
service           1.0.0    java-archive
```

No lodash package was identified.

ESTABLISH

The final-JAR scan gives:

```
Known build input             Final artefact scan

jackson-databind 2.19.4       2.19.4 identified
commons-codec 1.17.1          1.17.1 identified
lodash 4.17.21                 not identified
```

But the scan also reveals `commons-codec:1.18.0`, which we have not yet accounted for.

WHY WE NEED TO DO THIS

A scanner finding is evidence, but we should independently verify surprising results where possible.

Syft reported a second commons-codec version. Before explaining why, we first need to establish that `1.18.0` really exists as a physical nested JAR rather than being a duplicate identification or metadata artefact.

HOW WE'RE GOING TO DO IT

We inspect the executable JAR directly for any archive whose name contains `commons-codec`.

RUN

```
unzip -l service/target/service-1.0.0.jar \
```

```
  | grep 'commons-codec'
```

OBSERVED OUTPUT

```
373045  01-24-2025 14:02   BOOT-INF/lib/commons-codec-1.18.0.jar
```

ESTABLISH

The application genuinely contains two versions of commons-codec code:

```
BOOT-INF/lib/normalizer-1.0.0.jar
    relocated commons-codec 1.17.1

BOOT-INF/lib/commons-codec-1.18.0.jar
    ordinary commons-codec 1.18.0
```

WHY WE NEED TO DO THIS

We now have an apparent contradiction:

- `normalizer` resolved and embedded `commons-codec:1.17.1`.
- the final service also contains `commons-codec:1.18.0`.

This is where dependency resolution and physical packaging diverge. We need Maven's view of the service graph to explain the extra version.

HOW WE'RE GOING TO DO IT

We run `dependency:tree` for `service` with `-am` so the reactor's `normalizer` module participates.

`-Dverbose` asks the dependency plugin to include mediation information such as the version that was managed from another value. The important phrase in the output is `version managed from 1.17.1`.

RUN

```
mvn -pl service -am dependency:tree \
  -Dincludes=commons-codec:commons-codec \
  -Dverbose
```

OBSERVED OUTPUT

Normalizer:

```
[INFO] dev.noregressions.trace:normalizer:jar:1.0.0
[INFO] \- commons-codec:commons-codec:jar:1.17.1:compile
```

Service:

```
[INFO] dev.noregressions.trace:service:jar:1.0.0
[INFO] \- dev.noregressions.trace:normalizer:jar:1.0.0:compile
[INFO]    \- commons-codec:commons-codec:jar:1.18.0:compile (version managed f
```

ESTABLISH

Normalizer build:

```
commons-codec 1.17.1
        ↓
Shade
        ↓
1.17.1 embedded inside normalizer
```

Service resolution:

```
normalizer
    └── commons-codec 1.17.1
                ↓
        dependency management
                ↓
        commons-codec 1.18.0
```

Maven dependency management can change the transitive dependency selected for `service`; it cannot retroactively replace code already embedded inside the shaded `normalizer` JAR.

Therefore:

```
dependency graph != complete physical software inventory
```

WHY WE NEED TO DO THIS

So far we have compared resolver output with artefact inspection. Now we want to see how those different viewpoints affect a formal SBOM.

The first SBOM will be generated from Maven's dependency model. That means it should describe what Maven believes the project depends on, not necessarily everything physically embedded inside transformed artefacts.

HOW WE'RE GOING TO DO IT

We invoke the CycloneDX Maven Plugin directly by its full Maven coordinate and goal:

`org.cyclonedx:cyclonedx-maven-plugin:2.9.3:makeBom`

This avoids relying on the plugin already being configured in the project POM.

`-pl service -am` runs the goal for the service and required reactor modules.

`-DoutputFormat=json` asks the plugin to emit CycloneDX JSON rather than XML.

CycloneDX is the SBOM data standard here; the plugin is the producer.

RUN

```
mvn -pl service -am \
  org.cyclonedx:cyclonedx-maven-plugin:2.9.3:makeBom \
  -DoutputFormat=json
```

OBSERVED OUTPUT

```
root BOM
    0 components
    target/bom.json

normalizer BOM
    1 component
    normalizer/target/bom.json

service BOM
    36 components
    service/target/bom.json
```

ESTABLISH

CycloneDX generated a separate SBOM from the Maven dependency model of each reactor project.

WHY WE NEED TO DO THIS

The same component can be resolved differently in different modules. We want to see whether the module-level SBOMs preserve that distinction.

In particular, we expect the normalizer BOM to say `commons-codec:1.17.1` while the service BOM follows the service's managed dependency graph and says `1.18.0`.

HOW WE'RE GOING TO DO IT

`compare-sboms.sh` reads the two generated CycloneDX JSON files and prints only our tracer components.

Internally it uses `jq`, a JSON query tool, to select components by name and emit three fields: package name, version, and PURL.

A PURL (Package URL) is a standard package identifier such as:

```
pkg:maven/commons-codec/commons-codec@1.17.1
```

The script exists so the comparison is repeatable and does not depend on remembering the `jq` expression.

RUN

```
./scripts/compare-sboms.sh
```

OBSERVED OUTPUT

```
=== normalizer BOM ===

commons-codec   1.17.1  pkg:maven/commons-codec/commons-codec@1.17.1?type=jar

=== service BOM ===

jackson-databind    2.19.4  pkg:maven/com.fasterxml.jackson.core/jackson-datab
normalizer  1.0.0   pkg:maven/dev.noregressions.trace/normalizer@1.0.0?type=ja
commons-codec   1.18.0  pkg:maven/commons-codec/commons-codec@1.18.0?type=jar
```

ESTABLISH

The normalizer SBOM contains:

```
commons-codec 1.17.1
```

The service SBOM contains:

```
normalizer        1.0.0
commons-codec     1.18.0
jackson-databind  2.19.4
```

The service's Maven-generated SBOM does not include the `commons-codec:1.17.1` code already embedded inside `normalizer-1.0.0.jar`. It also has no knowledge of lodash.

WHY WE NEED TO DO THIS

Comparing Maven console output with Syft console output is useful, but the tools are not producing the same kind of document.

To isolate the effect of **evidence source**, we want both tools to emit the same SBOM format for the same application.

HOW WE'RE GOING TO DO IT

Syft supports CycloneDX JSON output.

`-o cyclonedx-json=...` selects the output format and file in one argument. The input remains the finished Spring Boot JAR, so Syft is still deriving its inventory from the artefact rather than from Maven's model.

RUN

```
mkdir -p trace-output

syft service/target/service-1.0.0.jar \
  -o cyclonedx-json=trace-output/service-syft.cdx.json
```

OBSERVED OUTPUT

```
✔ Indexed file system
✔ Cataloged contents
   ├── ✔ Packages [34 packages]
   ├── ✔ Executables [0 executables]
   ├── ✔ File digests [1 files]
```

Generated:

```
trace-output/service-syft.cdx.json
```

ESTABLISH

We now have two CycloneDX SBOMs for the same application:

```
service/target/bom.json
    generated from Maven's dependency model

trace-output/service-syft.cdx.json
    generated from finished-artefact inspection
```

WHY WE NEED TO DO THIS

This is the key SBOM comparison in the lab.

If the inventories differ now, the difference cannot be blamed on SBOM format: both documents are CycloneDX. The difference comes from **where in the supply chain the inventory was observed and what evidence each producer used**.

HOW WE'RE GOING TO DO IT

`compare-service-sboms.sh` reads the Maven-generated and Syft-generated CycloneDX JSON documents and prints the same tracer fields from both.

Again, the script uses `jq` internally so the comparison is deterministic and easy to rerun.

RUN

```
./scripts/compare-service-sboms.sh
```

OBSERVED OUTPUT

```
=== Maven-generated service SBOM ===

commons-codec   1.18.0  pkg:maven/commons-codec/commons-codec@1.18.0?type=jar
jackson-databind    2.19.4  pkg:maven/com.fasterxml.jackson.core/jackson-datab
normalizer  1.0.0   pkg:maven/dev.noregressions.trace/normalizer@1.0.0?type=ja

=== Syft-generated service SBOM ===

commons-codec   1.17.1  pkg:maven/commons-codec/commons-codec@1.17.1
commons-codec   1.18.0  pkg:maven/commons-codec/commons-codec@1.18.0
jackson-databind    2.19.4  pkg:maven/com.fasterxml.jackson.core/jackson-datab
normalizer  1.0.0   pkg:maven/dev.noregressions.trace/normalizer@1.0.0
```

ESTABLISH

This is an apples-to-apples comparison:

```
Maven-generated CycloneDX SBOM
    commons-codec 1.18.0
    jackson-databind 2.19.4
    normalizer 1.0.0

Syft-generated CycloneDX SBOM
    commons-codec 1.17.1
    commons-codec 1.18.0
```

```
    jackson-databind 2.19.4
    normalizer 1.0.0
```

Both describe the same application and use the same SBOM standard, but they contain different software inventories because they use different evidence sources.

```
Maven/CycloneDX
    resolved dependency model
        ↓
    commons-codec 1.18.0

Syft/CycloneDX
    finished JAR
        ↓
    commons-codec 1.17.1
    commons-codec 1.18.0
```

Therefore:

```
SBOM format does not determine inventory completeness.

Where and how the SBOM is generated matters.
```

WHY WE NEED TO DO THIS

The application JAR is not the whole product we deploy. A container image adds a JRE, operating-system packages, native libraries, shell utilities, certificates, and other runtime material.

An application-level SBOM can therefore be correct while still describing only part of the software ultimately shipped.

The container image is the outermost boundary in this exercise.

HOW WE'RE GOING TO DO IT

`image-trace.sh` performs three related actions:

1. It builds the Docker image from the project's `Dockerfile` using the tag `registry.example.com/checkout-service:release-123`.
2. It asks Docker for the resulting local image identity, including the repository digest. A tag is a mutable name; the digest identifies the built OCI image content.
3. It asks Syft to catalog the **container image**, not just the application JAR, and writes a CycloneDX image SBOM before printing the tracer components.

The Docker build output also shows the digest to which the mutable base-image tag `eclipse-temurin:21-jre-jammy` resolved for this build. This is useful evidence because the base image itself is part of the product's software supply chain.

RUN

```
./scripts/image-trace.sh
```

OBSERVED BUILD IDENTITY

The Dockerfile requests:

```
eclipse-temurin:21-jre-jammy
```

During this build Docker resolved the base image to:

```
eclipse-temurin:21-jre-jammy@sha256:8208b3a3733529f42c4b565bfcbf16d914eb628f3e
```

The application image was named:

```
registry.example.com/checkout-service:release-123
```

The local repository digest was:

```
registry.example.com/checkout-service@sha256:486526852f765f196f97765b17a3f1828
```

OBSERVED IMAGE SCAN

```
Packages      179
Executables   837
```

Tracer components:

```
commons-codec     1.17.1
commons-codec     1.18.0
jackson-databind  2.19.4
normalizer        1.0.0
```

No lodash component was identified.

ESTABLISH

The application-level inventory survived the JAR → image transition:

```
Service JAR
    commons-codec 1.17.1
    commons-codec 1.18.0
    jackson-databind 2.19.4
    normalizer 1.0.0

        ↓ Docker build

Container image
    commons-codec 1.17.1
    commons-codec 1.18.0
    jackson-databind 2.19.4
    normalizer 1.0.0
```

But the product inventory expanded:

```
service JAR
    34 packages

container image
```

```
    179 packages
    837 executables
```

The container includes substantially more software than the application artefact alone because it also contains the runtime environment and base operating-system contents.

This demonstrates:

```
application SBOM != container/product SBOM
```

The container image is the outermost deployable artefact in this lab.

Starting from three ordinary dependency declarations, we followed them through resolution, transformation, packaging, SBOM generation, and containerisation.

The important findings are:

```
1. A source declaration is not the same thing as a resolved dependency.

2. A resolved dependency is not proof of what was physically shipped.

3. Transformation can preserve code while destroying or preserving component i

4. software presence != software identifiability

5. A consuming Maven dependency graph can omit software already embedded insid

6. Two CycloneDX SBOMs for the same application can legitimately contain diffe

7. Bundled frontend dependencies can be present in shipped code while becoming

8. The container image contains a larger software universe than the applicatio
```

The most important conceptual point is that every inventory is an observation made from a particular evidence source at a particular point in the supply chain.

```
source configuration
        ↓
resolver model
        ↓
build transformation
        ↓
application artefact
        ↓
SBOM producer
        ↓
container image
```

Those views should be compared, not assumed to be interchangeable.

A separate reverse-provenance exercise should examine the different problem of starting from a binary/container and determining whether it is possible to identify the corresponding source repository and exact source commit.

PAYARA + MVNPM SUPPLY CHAIN TRACE LAB

This lab follows four different kinds of software through one build and into one running container:

- `commons-lang3` — an ordinary Maven application dependency that survives as a discrete JAR in the WAR.
- `lodash-es` — npm-origin software consumed as a Maven **plugin dependency**, transformed by esbuild into browser JavaScript.
- `jakarta.jakartaee-web-api` — a Maven `provided` dependency used to compile the application but deliberately omitted from the WAR.
- Payara, Jakarta runtime libraries, the JDK, and operating-system packages — software supplied by the final container image.

The point is not to teach Maven, mvnpm, Payara, Docker, or SBOM basics. The point is to see **what evidence exists at each stage, which dependency domain a component belongs to, what survives transformation, and what different inventory tools can legitimately know.**

The pattern throughout is:

Look → Run → Observe → Establish

The exercise ends at the container-image boundary.

WHY WE NEED TO DO THIS

Supply-chain tracing is only useful if we know which build produced the artefacts we inspect. Old Maven output, generated JavaScript, SBOMs, or trace output can make later evidence ambiguous.

HOW WE'RE GOING TO DO IT

Remove generated trace output and run Maven's normal clean lifecycle.

RUN

```
rm -rf trace-output
mvn clean
```

ESTABLISH

Generated Maven, frontend, WAR, and SBOM output from earlier runs is removed. Source configuration remains unchanged.

WHY WE NEED TO DO THIS

Before asking Maven, Syft, or an SBOM producer what exists, we need the literal declarations the developer gave the build.

This scenario deliberately uses three different dependency paths:

```
ordinary application dependency
    commons-lang3

provided application dependency
    jakarta.jakartaee-web-api
```

```
build-plugin dependency
    lodash-es
```

Those declarations do not have the same resolution or packaging semantics.

HOW WE'RE GOING TO DO IT

Inspect the relevant sections of `pom.xml` directly.

commons-lang3

RUN

```
grep -n -A5 -B2 'commons-lang3' pom.xml
```

OBSERVED OUTPUT

```
16-        <maven.compiler.release>21</maven.compiler.release>
17-        <jakartaee.version>11.0.0</jakartaee.version>
18:        <commons-lang3.version>3.18.0</commons-lang3.version>
19-        <lodash-es.version>4.17.21</lodash-es.version>
20-        <esbuild-maven-plugin.version>2.0.0</esbuild-maven-plugin.version>
21-    </properties>
22-
23-    <dependencies>
--
31-        <dependency>
32-            <groupId>org.apache.commons</groupId>
33:            <artifactId>commons-lang3</artifactId>
34:            <version>${commons-lang3.version}</version>
35-        </dependency>
36-    </dependencies>
37-
38-    <build>
39-        <finalName>${project.artifactId}-${project.version}</finalName>
```

Jakarta EE Web API

RUN

```
grep -n -A5 -B2 'jakarta.jakartaee-web-api' pom.xml
```

OBSERVED OUTPUT

```
24-        <dependency>
25-            <groupId>jakarta.platform</groupId>
26:            <artifactId>jakarta.jakartaee-web-api</artifactId>
27-            <version>${jakartaee.version}</version>
28-            <scope>provided</scope>
29-        </dependency>
```

```
30-
31-         <dependency>
```

lodash-es

RUN

```
grep -n -A8 -B4 'lodash-es' pom.xml
```

OBSERVED OUTPUT

```
15-         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
16-         <maven.compiler.release>21</maven.compiler.release>
17-         <jakartaee.version>11.0.0</jakartaee.version>
18-         <commons-lang3.version>3.18.0</commons-lang3.version>
19:         <lodash-es.version>4.17.21</lodash-es.version>
20-         <esbuild-maven-plugin.version>2.0.0</esbuild-maven-plugin.version>
21-     </properties>
22-
23-     <dependencies>
24-         <dependency>
25-             <groupId>jakarta.platform</groupId>
26-             <artifactId>jakarta.jakartaee-web-api</artifactId>
27-             <version>${jakartaee.version}</version>
--
67-             </configuration>
68-             <dependencies>
69-                 <dependency>
70-                     <groupId>org.mvnpm</groupId>
71:                     <artifactId>lodash-es</artifactId>
72:                     <version>${lodash-es.version}</version>
73-                 </dependency>
74-             </dependencies>
75-         </plugin>
76-
77-         <plugin>
78-             <groupId>org.apache.maven.plugins</groupId>
79-             <artifactId>maven-war-plugin</artifactId>
80-             <version>3.4.0</version>
```

ESTABLISH

The source configuration asks Maven to treat the three tracers differently:

```
commons-lang3:3.18.0
    normal project dependency

jakarta.jakartaee-web-api:11.0.0
    project dependency
    Maven scope: provided

lodash-es:4.17.21
```

```
    dependency of esbuild-maven-plugin
    not a normal WAR project dependency
```

These are declarations. We have not yet proved how Maven resolves them or what reaches the finished artefact.

WHY WE NEED TO DO THIS

The source configuration only describes intent. The interesting evidence appears after Maven resolves dependencies, constructs the plugin execution realm, runs esbuild, and packages the generated output into the WAR.

HOW WE'RE GOING TO DO IT

Use the project's normal build wrapper and then verify the generated browser assets and WAR directly.

RUN

```
./scripts/build.sh
```

Then verify the outputs:

```
find target/generated-web -maxdepth 2 -type f -print
ls -lh target/payara-mvnpm-trace-lab-1.0.0.war
```

OBSERVED OUTPUT

```
target/generated-web/assets/app.js.map
target/generated-web/assets/app.js
```

```
-rw-r--r--@ 1 spoole  staff   638K 22 Aug 09:44 target/payara-mvnpm-trace-lab-
```

ESTABLISH

The normal build produced:

```
esbuild output
    target/generated-web/assets/app.js
    target/generated-web/assets/app.js.map

deployable application
    target/payara-mvnpm-trace-lab-1.0.0.war
```

The build succeeded, but we still need to determine which dependency evidence belongs to the application graph and which belongs to the build itself.

WHY WE NEED TO DO THIS

`commons-lang3` is our control case. It is declared as an ordinary project dependency, so it should appear in Maven's normal project dependency graph.

HOW WE'RE GOING TO DO IT

Ask Maven's Dependency Plugin for the resolved project graph and filter it to `commons-lang3`.

RUN

```
mvn dependency:tree \
  -Dincludes=org.apache.commons:commons-lang3
```

OBSERVED OUTPUT

```
[INFO] --- dependency:3.7.0:tree (default-cli) @ payara-mvnpm-trace-lab ---
[INFO] dev.noregressions.trace:payara-mvnpm-trace-lab:war:1.0.0
[INFO] \- org.apache.commons:commons-lang3:jar:3.18.0:compile
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
```

ESTABLISH

```
pom.xml
    commons-lang3:3.18.0

        ↓ Maven project dependency resolution

project dependency graph
    commons-lang3:3.18.0
    scope: compile
```

Maven sees `commons-lang3:3.18.0` as normal application software.

WHY WE NEED TO DO THIS

`lodash-es` is declared under `esbuild-maven-plugin`, not under the project's ordinary `<dependencies>` section.

We want to prove that Maven's normal project dependency graph therefore does not treat it like `commons-lang3`.

HOW WE'RE GOING TO DO IT

Run the same project dependency-tree query, filtered to the mvnpm coordinate.

RUN

```
mvn dependency:tree \
  -Dincludes=org.mvnpm:lodash-es
```

OBSERVED OUTPUT

```
[INFO] --- dependency:3.7.0:tree (default-cli) @ payara-mvnpm-trace-lab ---
[INFO] ------------------------------------------------------------------------
```

```
[INFO] BUILD SUCCESS
```

No `lodash-es` dependency appears beneath the project.

ESTABLISH

```
pom.xml
    esbuild-maven-plugin
        └── org.mvnpm:lodash-es:4.17.21

        X not part of

Maven project dependency graph
```

A normal `dependency:tree` for the WAR does not contain `lodash-es`.

This does not mean the build did not use it. Maven maintains different dependency domains for the project being built and the plugins that perform the build.

WHY WE NEED TO DO THIS

The project dependency graph does not contain `lodash-es`. The next obvious question is whether Maven's plugin dependency reporting exposes it.

HOW WE'RE GOING TO DO IT

Ask the Maven Dependency Plugin to resolve the esbuild Maven plugin and its published dependency set.

RUN

```
mvn dependency:resolve-plugins \
  -DincludeArtifactIds=esbuild-maven-plugin
```

OBSERVED OUTPUT

Representative excerpt:

```
[INFO] The following plugins have been resolved:
[INFO]    io.mvnpm:esbuild-maven-plugin:maven-plugin:2.0.0:runtime
[INFO]       io.mvnpm:esbuild-maven-plugin:jar:2.0.0
[INFO]       io.mvnpm:esbuild-java-plugin-sass:jar:2.1.1
[INFO]       io.mvnpm:esbuild-java:jar:2.1.1
[INFO]       org.jboss.logging:jboss-logging:jar:3.6.1.Final
[INFO]       org.mvnpm:esbuild:jar:0.25.10
[INFO]       io.mvnpm:importmap:jar:1.0.11
...
[INFO] BUILD SUCCESS
```

`org.mvnpm:lodash-es:4.17.21` was **not** listed.

ESTABLISH

```
project dependency tree
    lodash-es absent

resolve-plugins report
    esbuild-maven-plugin present
    published plugin dependencies present
    lodash-es absent

POM
    lodash-es explicitly added under this project's
    esbuild-maven-plugin <dependencies>
```

`resolve-plugins` is not sufficient evidence for this project-specific plugin dependency. We need to inspect the actual execution realm Maven constructs for the plugin.

WHY WE NEED TO DO THIS

A project can add dependencies to a Maven plugin. Maven resolves those dependencies for the plugin's isolated execution realm rather than for the application's normal dependency graph.

That means build-time software can affect the bytes we ship while remaining absent from `dependency:tree`.

HOW WE'RE GOING TO DO IT

Run the `generate-resources` phase with Maven debug logging enabled and filter the plugin-realm construction to `lodash-es`.

RUN

```
mvn -X generate-resources 2>&1 \
  | grep 'org.mvnpm:lodash-es'
```

OBSERVED OUTPUT

```
[DEBUG]    org.mvnpm:lodash-es:jar:4.17.21:runtime
[DEBUG]   Included: org.mvnpm:lodash-es:jar:4.17.21
```

ESTABLISH

```
project dependency graph
    lodash-es absent

resolve-plugins report
    lodash-es absent

actual esbuild plugin execution realm
    lodash-es:4.17.21 present
```

The build can therefore use `lodash-es` even though a normal application dependency tree never reports it.

This is the first important supply-chain distinction in the lab:

```
application dependency graph
    !=
complete set of software that can influence the build
```

WHY WE NEED TO DO THIS

Finding `lodash-es` in the plugin execution realm only proves that the package was available to esbuild. It does not prove that lodash code contributed to the application output.

We need evidence from the generated artefact.

HOW WE'RE GOING TO DO IT

The build produced `app.js` and `app.js.map`. The source map records source modules represented in the generated JavaScript.

RUN

```
find target/generated-web -maxdepth 2 -type f -print
```

OBSERVED OUTPUT

```
target/generated-web/assets/app.js.map
target/generated-web/assets/app.js
```

ESTABLISH

The esbuild execution produced browser-facing application code and a source map.

```
plugin execution
        ↓ esbuild
target/generated-web/assets/app.js
target/generated-web/assets/app.js.map
```

We still need to prove that lodash contributed to those bytes.

WHY WE NEED TO DO THIS

The plugin execution realm tells us `lodash-es:4.17.21` was available. The source map lets us independently establish whether lodash modules contributed to the output.

HOW WE'RE GOING TO DO IT

Read the source map's `sources` array and filter it to lodash modules.

RUN

```
jq -r '.sources[]' target/generated-web/assets/app.js.map \
  | grep 'lodash'
```

OBSERVED OUTPUT

```
../../../../node_modules/lodash-es/_freeGlobal.js
../../../../node_modules/lodash-es/_root.js
../../../../node_modules/lodash-es/_Symbol.js
../../../../node_modules/lodash-es/_getRawTag.js
../../../../node_modules/lodash-es/_objectToString.js
../../../../node_modules/lodash-es/_baseGetTag.js
../../../../node_modules/lodash-es/isObjectLike.js
../../../../node_modules/lodash-es/isSymbol.js
../../../../node_modules/lodash-es/_arrayMap.js
../../../../node_modules/lodash-es/isArray.js
../../../../node_modules/lodash-es/_baseToString.js
../../../../node_modules/lodash-es/toString.js
../../../../node_modules/lodash-es/_baseSlice.js
../../../../node_modules/lodash-es/_castSlice.js
../../../../node_modules/lodash-es/_hasUnicode.js
../../../../node_modules/lodash-es/_asciiToArray.js
../../../../node_modules/lodash-es/_unicodeToArray.js
../../../../node_modules/lodash-es/_stringToArray.js
../../../../node_modules/lodash-es/_createCaseFirst.js
../../../../node_modules/lodash-es/upperFirst.js
../../../../node_modules/lodash-es/_arrayReduce.js
../../../../node_modules/lodash-es/_basePropertyOf.js
../../../../node_modules/lodash-es/_deburrLetter.js
../../../../node_modules/lodash-es/deburr.js
../../../../node_modules/lodash-es/_asciiWords.js
../../../../node_modules/lodash-es/_hasUnicodeWord.js
../../../../node_modules/lodash-es/_unicodeWords.js
../../../../node_modules/lodash-es/words.js
../../../../node_modules/lodash-es/_createCompounder.js
../../../../node_modules/lodash-es/_escapeHtmlChar.js
../../../../node_modules/lodash-es/escape.js
../../../../node_modules/lodash-es/startCase.js
```

ESTABLISH

```
plugin execution evidence
    org.mvnpm:lodash-es:4.17.21

        ↓ esbuild

source-map evidence
    lodash-es modules contributed to app.js
```

The source-map paths establish the package identity `lodash-es`; the version comes from the plugin-execution evidence.

We have now proved that lodash did not merely exist in Maven's build environment. Its code contributed to the generated application JavaScript.

WHY WE NEED TO DO THIS

Now deliberately throw away the Maven build context.

We know `lodash-es` contributed code. The question is whether an artefact scanner can recover the npm package identity from only the generated frontend output.

HOW WE'RE GOING TO DO IT

Give Syft only `target/generated-web`. It receives no POM, plugin realm, mvnpm coordinate, or original package tree.

RUN

```
syft target/generated-web
```

OBSERVED OUTPUT

```
✔ Indexed file system
✔ Cataloged contents
   ├── ✔ Packages [0 packages]
   └── ✔ Executables [0 executables]

WARN no explicit name and version provided for directory source, deriving arti

No packages discovered
```

ESTABLISH

```
build/plugin evidence
    lodash-es:4.17.21

source-map evidence
    lodash-es modules contributed code

artefact-only Syft scan
    0 packages
    lodash-es not identified
```

This is not evidence that lodash code is absent.

It demonstrates:

```
software contribution != package identifiability
```

WHY WE NEED TO DO THIS

The generated browser bundle exists, but that does not yet prove it reached the deployable application.

At the same boundary we can compare the ordinary Java dependency with the transformed npm-origin dependency.

HOW WE'RE GOING TO DO IT

Inspect the finished WAR directly for the versioned `commons-lang3` JAR and generated browser assets.

RUN

```
unzip -l target/payara-mvnpm-trace-lab-1.0.0.war \
  | grep -E 'WEB-INF/lib/commons-lang3|assets/app\.js'
```

OBSERVED OUTPUT

```
702952  07-06-2025 18:43   WEB-INF/lib/commons-lang3-3.18.0.jar
44987   08-22-2026 09:44   assets/app.js.map
8110    08-22-2026 09:44   assets/app.js
```

ESTABLISH

Both paths reached the same deployable WAR:

```
commons-lang3:3.18.0
        ↓
WEB-INF/lib/commons-lang3-3.18.0.jar
    package boundary and version survive

lodash-es:4.17.21
        ↓ esbuild
assets/app.js + assets/app.js.map
    contributed code survives
    npm package boundary does not
```

The WAR therefore contains third-party software from both ecosystems, but only one still looks like an ordinary package.

WHY WE NEED TO DO THIS

We have manually proved that the WAR contains both the conventional Java library and lodash-derived JavaScript.

Now ask an independent artefact scanner what package identities it can recover from the finished WAR alone.

HOW WE'RE GOING TO DO IT

Give Syft only the completed WAR.

RUN

```
syft target/payara-mvnpm-trace-lab-1.0.0.war
```

OBSERVED OUTPUT

```
✔ Packages [2 packages]

NAME                    VERSION  TYPE
```

```
commons-lang3              3.18.0   java-archive
payara-mvnpm-trace-lab     1.0.0    java-archive
```

ESTABLISH

```
WAR physically contains
    commons-lang3 JAR
    lodash-derived JavaScript

Syft identifies
    commons-lang3:3.18.0
    payara-mvnpm-trace-lab:1.0.0

Syft does not identify
    lodash-es
```

The central artefact-level result is:

```
code can be present in a deployable artefact
without the originating package remaining identifiable
```

WHY WE NEED TO DO THIS

The Jakarta EE Web API follows a third path.

It is a project dependency, but Maven scope `provided` tells the build that the runtime environment is expected to supply it. We should therefore find it in Maven's dependency model but not packaged into `WEB-INF/lib`.

HOW WE'RE GOING TO DO IT

First inspect the WAR library directory.

RUN

```
unzip -l target/payara-mvnpm-trace-lab-1.0.0.war \
  | grep 'WEB-INF/lib/'
```

OBSERVED OUTPUT

```
0       08-22-2026 09:44   WEB-INF/lib/
702952  07-06-2025 18:43   WEB-INF/lib/commons-lang3-3.18.0.jar
```

No Jakarta EE API JAR is packaged.

Now ask Maven for the resolved Jakarta dependency.

RUN

```
mvn dependency:tree \
  -Dincludes=jakarta.platform:jakarta.jakartaee-web-api
```

OBSERVED OUTPUT

```
[INFO] --- dependency:3.7.0:tree (default-cli) @ payara-mvnpm-trace-lab ---
[INFO] dev.noregressions.trace:payara-mvnpm-trace-lab:war:1.0.0
[INFO] \- jakarta.platform:jakarta.jakartaee-web-api:jar:11.0.0:provided
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
```

ESTABLISH

```
Maven dependency model
    jakarta.jakartaee-web-api:11.0.0
    scope: provided

WAR
    jakarta.jakartaee-web-api absent
```

Combined, this establishes:

```
jakarta.jakartaee-web-api:11.0.0
    resolved by Maven
    used for compilation
    deliberately omitted from the WAR
    expected to be supplied by the runtime
```

We now have three fully distinct dependency paths:

```
commons-lang3
    project dependency
    -> packaged as a JAR

lodash-es
    plugin dependency
    -> transformed into browser code
    -> package identity not recovered from WAR

Jakarta EE API
    provided project dependency
    -> in Maven model
    -> absent from WAR
```

WHY WE NEED TO DO THIS

So far we have compared Maven resolution with physical artefact inspection.

Now we want to see what a formal SBOM generated from Maven's project model says about those same three tracers.

HOW WE'RE GOING TO DO IT

Invoke the CycloneDX Maven Plugin directly and emit JSON.

RUN

```
mvn org.cyclonedx:cyclonedx-maven-plugin:2.9.3:makeBom \
  -DoutputFormat=json
```

OBSERVED OUTPUT

```
[INFO] CycloneDX: Resolving Dependencies
[INFO] CycloneDX: Creating BOM version 1.6 with 27 component(s)
[INFO] CycloneDX: Writing and validating BOM (JSON): .../target/bom.json
[INFO]            attaching as payara-mvnpm-trace-lab-1.0.0-cyclonedx.json
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
```

The plugin also emitted schema-keyword warnings during validation. They did not prevent BOM generation.

Now inspect the three tracer identities.

RUN

```
jq -r '.components[] | [.name, .version, (.scope // "-")] | @tsv' \
  target/bom.json \
  | grep -E 'commons-lang3|jakarta.jakartaee-web-api|lodash-es'
```

OBSERVED OUTPUT

```
jakarta.jakartaee-web-api    11.0.0    required
commons-lang3                3.18.0    required
```

`lodash-es` is absent.

ESTABLISH

The Maven-generated CycloneDX SBOM contains:

```
commons-lang3:3.18.0              present
jakarta.jakartaee-web-api:11.0.0  present
lodash-es:4.17.21                 absent
```

This is a model-derived inventory. It follows Maven's application dependency model, not a literal inventory of the files physically contained in the WAR.

WHY WE NEED TO DO THIS

The previous result looks surprising:

```
Maven
    jakarta.jakartaee-web-api
    scope: provided

WAR
    component physically absent
```

```
CycloneDX Maven SBOM
    component scope: required
```

It would be easy to read `required` as meaning "physically included in this WAR." That is not what this result means.

HOW WE'RE GOING TO DO IT

Keep the scope models separate.

Maven's `provided` describes Maven classpath and packaging behaviour: the dependency is available for compilation but is expected to be supplied by the runtime.

CycloneDX's component scope is a different model. `required` means the component is required for the described system to operate; it is not a statement that the component's bytes are physically embedded in the WAR.

The CycloneDX Maven Plugin includes Maven `provided` dependencies by default and does not preserve Maven's `provided` label as a CycloneDX scope value.

ESTABLISH

```
Maven "provided"
    !=
CycloneDX "required"
    !=
physical presence in the WAR
```

The Maven-generated SBOM is therefore not evidence that `jakarta.jakartaee-web-api-11.0.0.jar` is inside the WAR.

This becomes important when we compare it with an SBOM generated from the physical artefact.

WHY WE NEED TO DO THIS

We now have one CycloneDX SBOM generated from Maven's dependency model.

To isolate the effect of **evidence source**, generate a second CycloneDX SBOM from the completed WAR itself.

The format stays the same:

```
same application
same CycloneDX format

Maven model -> SBOM
WAR bytes   -> SBOM
```

HOW WE'RE GOING TO DO IT

Ask Syft to inspect the WAR and emit CycloneDX JSON.

RUN

```
mkdir -p trace-output
```

```
syft target/payara-mvnpm-trace-lab-1.0.0.war \
  -o cyclonedx-json=trace-output/war-syft.cdx.json
```

OBSERVED OUTPUT

```
✔ Indexed file system
✔ Cataloged contents
   ├── ✔ Packages [2 packages]
   ├── ✔ File digests [1 files]
   └── ✔ Executables [0 executables]
```

Now inspect the exact tracer component names rather than grepping the complete JSON.

RUN

```
jq -r '
  .components[]
  | select(
      .name == "commons-lang3" or
      .name == "jakarta.jakartaee-web-api" or
      .name == "lodash-es" or
      .name == "payara-mvnpm-trace-lab"
    )
  | [.name, .version, (.scope // "-")]
  | @tsv
' trace-output/war-syft.cdx.json
```

OBSERVED OUTPUT

```
commons-lang3            3.18.0    -
payara-mvnpm-trace-lab   1.0.0     -
```

ESTABLISH

The WAR-derived CycloneDX inventory contains only the identities Syft can recover from the physical WAR:

```
commons-lang3:3.18.0              present
payara-mvnpm-trace-lab:1.0.0      present
jakarta.jakartaee-web-api         absent
lodash-es                         absent
```

WHY WE NEED TO DO THIS

This is the key SBOM comparison in the lab.

Both documents use CycloneDX. Both describe the same application. The difference is where their inventory evidence came from.

ESTABLISH

```
Maven-generated CycloneDX

    commons-lang3               3.18.0   required
    jakarta.jakartaee-web-api   11.0.0   required
    lodash-es                   absent


WAR-derived CycloneDX

    commons-lang3               3.18.0
    payara-mvnpm-trace-lab      1.0.0
    jakarta.jakartaee-web-api   absent
    lodash-es                   absent
```

The differences are meaningful:

```
Maven model
    knows the application declares a provided Jakarta EE dependency
    does not include the project-specific esbuild plugin dependency

WAR inspection
    sees the packaged application and commons-lang3
    does not see the provided Jakarta API
    cannot recover lodash-es after bundling
```

Therefore:

```
SBOM format does not determine inventory completeness.

Where and how the SBOM is generated matters.
```

A model-derived SBOM can include software that is required but not packaged in the artefact. An artefact-derived SBOM can identify what survives as package evidence but cannot necessarily reconstruct transformed build inputs.

WHY WE NEED TO DO THIS

The WAR is not the complete runtime product.

The application deliberately omitted Jakarta runtime APIs, and Payara itself brings a much larger body of server, JDK, and operating-system software.

HOW WE'RE GOING TO DO IT

Build the project image and start the container with the supplied wrapper.

RUN

```
./scripts/run.sh
```

OBSERVED OUTPUT

The Docker build used:

```
payara/server-web:7.2026.7
```

For this walkthrough, Docker resolved that base image to:

```
payara/server-web:7.2026.7@sha256:989bde76edb44118bad9dba6d2d7ce93b7f0c0aab357
```

The application WAR was copied to:

```
/opt/payara/deployments/trace.war
```

and the local image was named:

```
payara-mvnpm-trace-lab:local
```

ESTABLISH

The WAR has crossed into a concrete Payara runtime image.

The base-image digest is build-time evidence for this walkthrough, not a value that should be assumed stable for future runs of the mutable tag.

Starting the container does not by itself prove that Payara successfully deployed the application. We verify that next.

WHY WE NEED TO DO THIS

We need runtime evidence, not merely image-construction evidence.

A successful request proves that Payara deployed the WAR and that code relying on both the packaged Java dependency and server-provided Jakarta functionality can execute.

HOW WE'RE GOING TO DO IT

Call the servlet endpoint.

RUN

```
curl -sS 'http://localhost:8080/trace/api/info?name=runtime%20trace' | jq
```

OBSERVED OUTPUT

```
{
  "message": "Hello Runtime trace",
  "application": "payara-mvnpm-trace-lab",
  "javaLibrary": "commons-lang3",
  "server": "Payara"
}
```

ESTABLISH

```
WAR
    commons-lang3 packaged
```

```
    Jakarta EE API not packaged

        ↓ deployed to Payara

runtime
    servlet executes successfully
    commons-lang3 works
    Jakarta JSON-P works
```

The application is not merely present in the image. Payara successfully deployed and executed it.

The runtime supplies Jakarta capability that was deliberately absent from the WAR.

WHY WE NEED TO DO THIS

The running product contains much more software than the WAR.

The final image adds Payara itself, concrete Jakarta runtime libraries, a JDK, Ubuntu packages, utilities, and other transitive runtime software.

This is the outermost software boundary in the exercise.

HOW WE'RE GOING TO DO IT

Give Syft the completed local container image.

RUN

```
syft payara-mvnpm-trace-lab:local
```

OBSERVED OUTPUT

Syft catalogued:

```
589 packages
825 executables
5,424 file-metadata locations
```

Representative entries relevant to this trace included:

```
commons-lang3                3.18.0       java-archive
payara-mvnpm-trace-lab       1.0.0        java-archive
jakarta.json-api             2.1.0        java-archive
jakarta.servlet-api          6.1.0        java-archive
payara-api                   7.2026.7     java-archive
zulu21-jre                   21.0.11-3    deb
```

The image also contains a large set of Payara `7.2026.7` modules, Jakarta APIs and implementations, JDK packages, and Ubuntu packages.

`lodash-es` was still not identified as a package.

ESTABLISH

Crossing from WAR to container dramatically changes the visible software universe:

```
WAR scan
    2 identifiable packages

container scan
    589 packages
    825 executables
```

At the container boundary:

```
commons-lang3
    still identifiable

application WAR
    still identifiable

runtime-provided Jakarta software
    now identifiable as concrete runtime components

Payara
    visible

JDK and OS
    visible

lodash-es
    still not identifiable as a package
```

This also clarifies the earlier Maven `provided` result.

The Maven project declares the umbrella compile-time API:

```
jakarta.jakartaee-web-api:11.0.0
```

The running container contains the concrete runtime libraries supplied by Payara, for example:

```
jakarta.json-api:2.1.0
jakarta.servlet-api:6.1.0
```

Those are different evidence views of the runtime requirement.

The application artefact is therefore not the whole deployed software inventory:

```
application SBOM != container/product SBOM
```

Starting from three deliberately different dependency declarations, we followed them through Maven resolution, plugin execution, transformation, WAR packaging, two SBOM producers, runtime deployment, and the final container image.

The important findings are:

```
1. Maven's normal project dependency graph does not describe every
   dependency that can influence a build.

2. A project-specific Maven plugin dependency can be absent from both
   dependency:tree and resolve-plugins while still being present in the
```

```
   actual plugin execution realm.

3. Finding software in a build environment is not proof that its code
   reached the product. Build-output evidence is needed.

4. lodash-es:4.17.21 contributed code to app.js, but after esbuild
   transformed it, Syft could no longer recover lodash-es as a package
   from either the generated frontend or the WAR.

5. software contribution != package identifiability

6. Maven `provided`, CycloneDX `required`, and physical presence in a WAR
   describe different things and must not be treated as interchangeable
   scope semantics.

7. Two CycloneDX SBOMs for the same application can legitimately contain
   different inventories because one is generated from the Maven model
   and the other from the physical WAR.

8. The container image introduces an additional software universe:
   Payara, concrete Jakarta runtime libraries, the JDK, operating-system
   packages, and other runtime software that does not exist in the WAR.
```

The complete evidence chain is:

```
source configuration
        ↓
Maven project graph / plugin execution realm
        ↓
build transformation
        ↓
WAR
        ↓
SBOM producer and evidence source
        ↓
Payara runtime
        ↓
container image
```

The central lesson is the same as the first trace lab:

Every inventory is an observation made from a particular evidence source at a particular point in the supply chain.

Those observations should be compared, not assumed to be interchangeable.

MAVEN PLUGIN HIDDEN-CONTENT SUPPLY CHAIN TRACE LAB

This lab follows runtime capability that enters a Java application through **Maven plugin execution**, rather than through the application's normal dependency graph.

The tracers are:

- `trace-injector-maven-plugin` — executable build-time software attached to the Maven lifecycle.
- `trace-route-payload` — a transitive dependency of that plugin.
- `/hidden/build-info` — a route definition carried by the plugin dependency.
- `GeneratedTraceRoute` — Java source generated during the build.
- `META-INF/services/...TraceRoute` — generated ServiceLoader metadata that activates the class at runtime.

The pattern is:

Look → Run → Observe → Establish

WHAT MAVEN PLUGINS MEAN IN THIS LAB

Maven has more than one dependency domain.

`mvn dependency:tree` describes the application's **project dependencies**. Maven plugins are different: they are executable build-time software. Maven resolves a plugin and its own dependencies, creates a **plugin classloader realm**, and invokes the plugin goal during the build lifecycle.

So these are different evidence views:

```
application dependency graph
        !=
plugin dependency graph
        !=
plugin execution realm
```

A plugin can then transform the application by generating source, resources, metadata or other content. Once that content has been compiled and packaged, the final JAR may contain the resulting behaviour without retaining an obvious package boundary for the build-time software that introduced it.

The important trace boundary is therefore:

```
plugin execution realm
        ↓
build transformation
        ↓
application bytes
        ↓
runtime behaviour
```

In this lab, the application POM declares `trace-injector-maven-plugin`, but does **not** declare `trace-route-payload` as an application dependency. The payload is a transitive dependency of the plugin. During `generate-sources`, the plugin reads that payload and creates Java code plus ServiceLoader metadata. Maven then compiles and packages those generated files as ordinary application content.

WHY WE NEED TO DO THIS

Separate source-controlled content from previous generated build output.

RUN

```
./scripts/clean.sh
```

OBSERVED OUTPUT

The cleanup script completed successfully during the walkthrough.

ESTABLISH

The application can be rebuilt from the checked-in scenario sources and local Maven fixture repository.

WHY WE NEED TO DO THIS

Start with the Maven view most developers use when asking "what does this application depend on?"

RUN

```
mvn \
  -Dmaven.repo.local="$PWD/.maven-repo" \
  dependency:tree
```

OBSERVED OUTPUT

```
[INFO] --------< dev.noregressions.trace:maven-plugin-hidden-content >--------
[INFO] Building Maven Plugin Hidden Content Trace Lab 1.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------

[INFO] --- dependency:3.7.0:tree (default-cli) @ maven-plugin-hidden-content -
[INFO] dev.noregressions.trace:maven-plugin-hidden-content:jar:1.0.0

[INFO] BUILD SUCCESS
```

ESTABLISH

The application's normal Maven dependency graph is empty:

```
maven-plugin-hidden-content:1.0.0
```

Neither `trace-injector-maven-plugin` nor `trace-route-payload` appears as an application dependency.

WHY WE NEED TO DO THIS

The project dependency graph is not Maven's only dependency domain.

RUN

```
mvn \
  -Dmaven.repo.local="$PWD/.maven-repo" \
  dependency:resolve-plugins \
  -DincludeArtifactIds=trace-injector-maven-plugin
```

OBSERVED OUTPUT

```
[INFO] The following plugins have been resolved:

[INFO]    dev.noregressions.trace:trace-injector-maven-plugin:maven-plugin:1.0
[INFO]       dev.noregressions.trace:trace-injector-maven-plugin:jar:1.0.0
[INFO]       dev.noregressions.trace:trace-route-payload:jar:1.0.0

[INFO] BUILD SUCCESS
```

ESTABLISH

When we ask Maven about the **plugin** dependency domain, a second graph appears:

```
trace-injector-maven-plugin:1.0.0
        ↓
trace-route-payload:1.0.0
```

Maven knows about the payload, but not as an application dependency.

WHY WE NEED TO DO THIS

Resolver output tells us what Maven can resolve. Debug output tells us what Maven actually loads to execute the plugin.

RUN

```
mvn \
  -Dmaven.repo.local="$PWD/.maven-repo" \
  -X generate-sources 2>&1 \
  | grep -E 'trace-injector|trace-route-payload'
```

OBSERVED OUTPUT

```
[DEBUG] Goal:          dev.noregressions.trace:trace-injector-maven-plugin:1.0
[INFO] --- trace-injector:1.0.0:inject-route (inject-build-route) @ maven-plug
[DEBUG] dev.noregressions.trace:trace-injector-maven-plugin:jar:1.0.0
[DEBUG]    dev.noregressions.trace:trace-route-payload:jar:1.0.0:compile
[DEBUG] Created new class realm plugin>dev.noregressions.trace:trace-injector-
[DEBUG] Populating class realm plugin>dev.noregressions.trace:trace-injector-m
[DEBUG]   Included: dev.noregressions.trace:trace-injector-maven-plugin:jar:1.
[DEBUG]   Included: dev.noregressions.trace:trace-route-payload:jar:1.0.0
[DEBUG] Loading mojo dev.noregressions.trace:trace-injector-maven-plugin:1.0.0
```

ESTABLISH

trace-route-payload is not merely resolvable. It is actually present in the Maven **plugin ClassRealm** used to execute inject-route.

```
application dependency graph
        no payload

plugin execution realm
        plugin + payload
```

WHY WE NEED TO DO THIS

Now follow the build-time input across the transformation boundary into application source.

RUN

```
find target/generated-sources -type f -print
```

OBSERVED OUTPUT

```
target/generated-sources/trace-injector/dev/noregressions/trace/s04/generated/
```

Then:

```
sed -n '1,220p' \
  target/generated-sources/trace-injector/dev/noregressions/trace/s04/generate
```

OBSERVED OUTPUT

```
package dev.noregressions.trace.s04.generated;

import dev.noregressions.trace.s04.TraceRoute;

public final class GeneratedTraceRoute implements TraceRoute {

    @Override
    public String path() {
        return "/hidden/build-info";
    }

    @Override
    public String responseJson() {
        return "{\n  \"message\": \"This runtime endpoint came from a transiti
    }
}
```

ESTABLISH

The runtime route now exists as Java application source even though it did not originate in the application's checked-in Java source.

Its provenance is explicit:

```
origin       = trace-route-payload
introducedBy = trace-injector-maven-plugin
route        = /hidden/build-info
```

WHY WE NEED TO DO THIS

Generating a class does not make it execute. We need to see how the build connects it to the application.

RUN

```
find target/generated-resources -type f -print
```

OBSERVED OUTPUT

```
target/generated-resources/trace-injector/META-INF/trace-lab/plugin-injection.
target/generated-resources/trace-injector/META-INF/services/dev.noregressions.
```

Inspect the ServiceLoader registration:

```
cat \
  target/generated-resources/trace-injector/META-INF/services/dev.noregression
```

OBSERVED OUTPUT

```
dev.noregressions.trace.s04.generated.GeneratedTraceRoute
```

Inspect the provenance marker:

```
cat \
  target/generated-resources/trace-injector/META-INF/trace-lab/plugin-injectio
```

OBSERVED OUTPUT

```
plugin=trace-injector-maven-plugin
payload=trace-route-payload
route=/hidden/build-info
```

ESTABLISH

The build generated both:

```
GeneratedTraceRoute.class
        +
META-INF/services/...TraceRoute
```

The ServiceLoader descriptor makes the generated implementation discoverable by the source-defined application at runtime.

WHY WE NEED TO DO THIS

Generated build directories are intermediate evidence. The deployable JAR is the shipped artefact.

RUN

```
unzip -l target/maven-plugin-hidden-content-1.0.0.jar \
  | grep -E 'GeneratedTraceRoute|META-INF/services|plugin-injection'
```

OBSERVED OUTPUT

```
        0  08-22-2026 11:58   META-INF/services/
       88  08-22-2026 11:58   META-INF/trace-lab/plugin-injection.properties
       58  08-22-2026 11:58   META-INF/services/dev.noregressions.trace.s04.Tr
      803  08-22-2026 11:58   dev/noregressions/trace/s04/generated/GeneratedT
```

ESTABLISH

The generated implementation, activation metadata and provenance marker are all bytes in the final application JAR.

WHY WE NEED TO DO THIS

Generated Java source may be deleted after the build. The final JAR still has to carry the behaviour.

RUN

```
javap \
  -classpath target/maven-plugin-hidden-content-1.0.0.jar \
  -c -p \
  dev.noregressions.trace.s04.generated.GeneratedTraceRoute
```

OBSERVED OUTPUT

```
Compiled from "GeneratedTraceRoute.java"

public final class dev.noregressions.trace.s04.generated.GeneratedTraceRoute i

  public java.lang.String path();
    Code:
       0: ldc           #7                  // String /hidden/build-info
       2: areturn

  public java.lang.String responseJson();
    Code:
       0: ldc           #9                  // String {\n  \"message\": \"This
```

```
        2: areturn
}
```

ESTABLISH

The final application bytecode contains:

```
/hidden/build-info
trace-route-payload
trace-injector-maven-plugin
```

The runtime behaviour survived the transformation even though the build-time package boundary did not.

WHY WE NEED TO DO THIS

Now compare physical runtime capability with a package scanner's inventory.

RUN

```
syft target/maven-plugin-hidden-content-1.0.0.jar
```

OBSERVED OUTPUT

```
✔ Indexed file system   target/maven-plugin-hidden-content-1.0.0.jar
✔ Cataloged contents

   ├── ✔ Packages      [1 packages]
   ├── ✔ File digests  [1 files]
   ├── ✔ Executables   [0 executables]

NAME                          VERSION  TYPE
maven-plugin-hidden-content   1.0.0    java-archive
```

ESTABLISH

Syft sees the application archive, but does not identify either of the build-time components responsible for the generated behaviour:

```
trace-injector-maven-plugin   not identified
trace-route-payload           not identified
```

The bytes are present. Their build provenance is not recovered as package identity.

WHY WE NEED TO DO THIS

Compare the scanner view with an SBOM generated from Maven's project dependency model.

RUN

```
mvn \
  -Dmaven.repo.local="$PWD/.maven-repo" \
  org.cyclonedx:cyclonedx-maven-plugin:2.9.3:makeBom \
  -DoutputFormat=json
```

OBSERVED OUTPUT

```
[INFO] CycloneDX: Resolving Dependencies
[INFO] CycloneDX: Creating BOM version 1.6 with 0 component(s)
[INFO] CycloneDX: Writing and validating BOM (JSON): .../target/bom.json
[INFO] BUILD SUCCESS
```

Focused check:

```
jq -r '.components[]? | [.name, .version] | @tsv' target/bom.json \
  | grep -E 'trace-injector|trace-route-payload|maven-plugin-hidden-content' |
```

OBSERVED OUTPUT

```

```

No matching output.

ESTABLISH

The Maven-model CycloneDX SBOM contains **zero dependency components.** It does not identify the build plugin or its transitive payload.

At this point the evidence views are:

```
Maven dependency tree     → no plugin or payload
CycloneDX Maven SBOM      → no plugin or payload
Maven plugin resolution   → plugin + payload
Maven execution realm     → plugin + payload
final JAR                 → generated runtime behaviour present
Syft JAR scan             → application archive only
```

WHY WE NEED TO DO THIS

The final test is whether the generated, packaged bytes actually change runtime capability.

RUN

```
./scripts/run.sh
```

OBSERVED OUTPUT

```
Runtime started as PID 69945
Open:   http://localhost:8082/
Health: http://localhost:8082/health
```

ESTABLISH

The application starts successfully from the final JAR on port `8082`.

WHY WE NEED TO DO THIS

This is the control endpoint: behaviour deliberately present in the checked-in application source.

RUN

```
curl -sS http://localhost:8082/health | jq
```

OBSERVED OUTPUT

```
{
  "application": "maven-plugin-hidden-content",
  "status": "UP"
}
```

ESTABLISH

The source-defined application is running normally.

WHY WE NEED TO DO THIS

This closes the chain from plugin dependency to runtime behaviour.

RUN

```
curl -sS http://localhost:8082/hidden/build-info | jq
```

OBSERVED OUTPUT

```
{
  "message": "This runtime endpoint came from a transitive Maven plugin depend
  "origin": "trace-route-payload",
  "introducedBy": "trace-injector-maven-plugin",
  "route": "/hidden/build-info"
}
```

ESTABLISH

The endpoint exists at runtime and explicitly ties itself back to:

```
trace-route-payload
        ↓
trace-injector-maven-plugin
        ↓
```

```
generated Java + ServiceLoader metadata
        ↓
application JAR
        ↓
/hidden/build-info
```

---

1. **Maven has multiple dependency domains.** An empty application dependency tree does not mean no third-party software participates in producing the application.
2. **Maven plugins are executable supply-chain inputs.** The plugin and its transitive dependencies are loaded into a separate plugin ClassRealm and execute during the build.
3. **A transitive plugin dependency can influence shipped behaviour without becoming an application dependency.** `trace-route-payload` supplied data used to create runtime code, but never appeared in `mvn dependency:tree`.
4. **Build transformations can destroy obvious package identity.** The plugin and payload become generated Java bytecode and ServiceLoader metadata in the application JAR.
5. **Presence and provenance are different questions.** `javap` proves the behaviour is physically present. Syft identifies the application archive but does not recover the plugin or payload that caused those bytes to exist.
6. **A normal dependency-derived SBOM can omit causally important build software.** The CycloneDX Maven plugin produced zero dependency components even though two external build components participated in producing the JAR.
7. **The correct evidence depends on the question.** Project graph, plugin graph, plugin realm, generated source, packaged bytes, scanner output, SBOM and runtime each reveal different parts of the same supply-chain event.

The complete evidence chain is:

```
application POM
        ↓
trace-injector-maven-plugin
        ↓
trace-route-payload
        ↓
Maven plugin ClassRealm
        ↓
GeneratedTraceRoute.java
        +
META-INF/services registration
        ↓
compiled/package application JAR
        ↓
ServiceLoader
        ↓
GET /hidden/build-info
```

S05 — NODE NPM PREPACK SUPPLY CHAIN TRACE LAB

This lab follows a Node package from source, through an npm lifecycle hook, into a packed tarball, into `node_modules`, and finally into runtime behaviour.

The pattern is:

Look → Run → Observe → Establish

WHAT PREPACK MEANS IN THIS LAB

npm packages can execute lifecycle scripts while a distributable package is being created.

`trace-route-package` declares:

```
"scripts": {
  "prepack": "node scripts/generate-dist.js"
}
```

Its runtime entry point is:

```
"main": "dist/index.js"
```

but `dist/` is generated by the lifecycle script. The package also declares:

```
"files": [
  "dist"
]
```

so the tarball contains the generated runtime files while excluding the generator and its build input.

That creates an important supply-chain execution boundary:

```
package source
        ↓
package-owned lifecycle code executes
        ↓
generated distribution files
        ↓
packed npm artefact
```

The packed package is therefore not simply a copy of the source tree.

In this lab the complete chain is:

```
build-input/route.json
        ↓
npm prepack
        ↓
scripts/generate-dist.js
        ↓
dist/index.js
dist/prepack-evidence.json
        ↓
npm pack
```

```
        ↓
trace-route-package-1.0.0.tgz
        ↓
node_modules/trace-route-package
        ↓
GET /hidden/prepack-info
```

WHY WE NEED TO DO THIS

We need to distinguish checked-in package inputs from files created by the npm lifecycle.

RUN

```
./scripts/clean.sh
```

OBSERVED OUTPUT

The scenario was cleaned before the build. During the walkthrough, the pre-build file listing was not captured separately.

The supplied `clean.sh` removes:

```
node_modules/
package-lock.json
npm-repo/
trace-output/
packages/trace-route-package/dist/
.runtime.pid
.runtime.log
```

ESTABLISH

The scenario has an explicit clean state in which the package's generated `dist/` directory is absent.

The automated proof-check verifies this directly before packing.

WHY WE NEED TO DO THIS

The first execution boundary is `npm pack`: we want to see whether package-owned code executes before the tarball is assembled.

RUN

```
./scripts/build.sh
```

OBSERVED OUTPUT

```
== Pack trace-route-package ==

npm notice run trace-route-package@1.0.0 prepack
npm notice run node scripts/generate-dist.js
```

```
generated dist/index.js for /hidden/prepack-info
generated dist/prepack-evidence.json

npm notice
npm notice 📦  trace-route-package@1.0.0
npm notice Tarball Contents
npm notice 435B dist/index.js
npm notice 334B dist/prepack-evidence.json
npm notice 300B package.json
npm notice Tarball Details
npm notice name: trace-route-package
npm notice version: 1.0.0
npm notice filename: trace-route-package-1.0.0.tgz
npm notice package size: 525 B
npm notice unpacked size: 1.1 kB
npm notice shasum: 480d3319ffb7d64dd91cf5065d588c971075f6b0
npm notice integrity: sha512-SbqxnntqZlzVp[...]51seeyFTJ3cig==
npm notice total files: 3
npm notice

trace-route-package-1.0.0.tgz

== Install application dependencies from the packed tarball ==

added 1 package in 167ms

== Installed dependency tree ==

node-prepack-trace-lab@1.0.0 .../scenarios/S05-node-prepack
└── trace-route-package@1.0.0
```

ESTABLISH

`npm pack` did not just archive existing files. It first executed:

```
trace-route-package@1.0.0 prepack
        ↓
node scripts/generate-dist.js
```

That execution created:

```
dist/index.js
dist/prepack-evidence.json
```

and those generated files immediately became tarball content.

The exact tarball size, shasum and integrity digest are observations from this walkthrough, not invariants of the scenario.

WHY WE NEED TO DO THIS

After the lifecycle has run, the source workspace contains both original inputs and generated output.

RUN

```
find packages/trace-route-package -maxdepth 3 -type f -print | sort
```

OBSERVED OUTPUT

```
packages/trace-route-package/.DS_Store
packages/trace-route-package/build-input/route.json
packages/trace-route-package/dist/index.js
packages/trace-route-package/dist/prepack-evidence.json
packages/trace-route-package/package.json
packages/trace-route-package/scripts/generate-dist.js
```

ESTABLISH

After prepack, three classes of evidence coexist in the package workspace:

```
original build input
    build-input/route.json

executable build logic
    scripts/generate-dist.js

generated distribution output
    dist/index.js
    dist/prepack-evidence.json
```

.DS_Store is incidental local filesystem metadata and is not part of the scenario.

WHY WE NEED TO DO THIS

The distributable tarball is a different evidence view from the package source workspace.

RUN

```
tar -tzf npm-repo/trace-route-package-1.0.0.tgz
```

OBSERVED OUTPUT

```
package/dist/index.js
package/package.json
package/dist/prepack-evidence.json
```

ESTABLISH

The tarball contains the generated runtime output, but not:

```
scripts/generate-dist.js
build-input/route.json
```

The publication boundary has therefore preserved the **result** of the lifecycle execution while removing the generator and its original input.

WHY WE NEED TO DO THIS

Although the actual generator and input are absent, this lab deliberately writes a provenance marker into the generated output so we can trace the transformation.

RUN

```
tar -xOzf npm-repo/trace-route-package-1.0.0.tgz \
  package/dist/prepack-evidence.json | jq
```

OBSERVED OUTPUT


```
{
  "event": "npm-prepack-generated",
  "package": "trace-route-package",
  "version": "1.0.0",
  "generatedBy": "npm lifecycle prepack -> scripts/generate-dist.js",
  "message": "This runtime route was generated by an npm prepack lifecycle scr
  "origin": "trace-route-package build input",
  "route": "/hidden/prepack-info"
}
```


ESTABLISH

The distributed artefact retains an explicit statement that its runtime content was generated by:

```
npm lifecycle prepack
        ↓
scripts/generate-dist.js
```

Without such retained metadata, the tarball itself would preserve the generated result but provide much less evidence about how it was produced.

WHY WE NEED TO DO THIS

Installation creates another evidence boundary. We want to see what survives into `node_modules`.

RUN

```
find node_modules/trace-route-package -maxdepth 3 -type f -print | sort
```

OBSERVED OUTPUT

```
node_modules/trace-route-package/dist/index.js
node_modules/trace-route-package/dist/prepack-evidence.json
```

```
node_modules/trace-route-package/package.json
```

ESTABLISH

The installed package mirrors the packed artefact. It contains generated runtime code, generated provenance metadata and package metadata, but not the generator or original build input.

RUN

```
cat node_modules/trace-route-package/dist/prepack-evidence.json | jq
```

OBSERVED OUTPUT


```
{
  "event": "npm-prepack-generated",
  "package": "trace-route-package",
  "version": "1.0.0",
  "generatedBy": "npm lifecycle prepack -> scripts/generate-dist.js",
  "message": "This runtime route was generated by an npm prepack lifecycle scr
  "origin": "trace-route-package build input",
  "route": "/hidden/prepack-info"
}
```


ESTABLISH

The generated provenance marker survives unchanged from tarball to installed package.

RUN

```
npm ls --all
```

OBSERVED OUTPUT

```
node-prepack-trace-lab@1.0.0 .../scenarios/S05-node-prepack
└── trace-route-package@1.0.0
```

ESTABLISH

npm identifies the dependency relationship, but package identity alone does not explain how the package's runtime files were produced.

RUN

```
npm sbom --sbom-format cyclonedx > trace-output/npm-sbom.json
```

Then:

```
jq -r '.components[]? | [.name, .version] | @tsv' trace-output/npm-sbom.json \
  | grep -E 'node-prepack-trace-lab|trace-route-package' || true
```

OBSERVED OUTPUT

```
trace-route-package 1.0.0
```

ESTABLISH

npm's CycloneDX SBOM identifies `trace-route-package@1.0.0`, but that component identity does not describe:

```
prepack
    ↓
generate-dist.js
    ↓
dist/index.js
```

The SBOM answers **what package is present.** It does not, in this view, answer **how the package's shipped runtime bytes were produced.**

RUN

```
syft node_modules/trace-route-package
```

OBSERVED OUTPUT

```
✔ Indexed file system  node_modules/trace-route-package
✔ Cataloged contents

   ├── ✔ Packages      [0 packages]
   └── ✔ Executables   [0 executables]

WARN no explicit name and version

No packages discovered
```

ESTABLISH

When given only the isolated installed package directory, Syft did not identify an npm package in this walkthrough.

RUN

```
syft dir:.
```

OBSERVED OUTPUT

```
✔ Indexed file system  .
✔ Cataloged contents

   ├── ✔ Packages        [2 packages]
   ├── ✔ Executables     [0 executables]
   ├── ✔ File digests    [1 files]
   └── ✔ File metadata   [1 locations]

NAME                     VERSION  TYPE
node-prepack-trace-lab   1.0.0    npm
trace-route-package      1.0.0    npm
```

ESTABLISH

With the surrounding project context, Syft recovers both npm package identities.

```
isolated node_modules package
        → 0 packages

whole project
        → 2 npm packages
```

Neither package inventory view, by itself, exposes the `prepack` transformation that generated the runtime code.

RUN

```
./scripts/run.sh
```

OBSERVED OUTPUT

```
Runtime started as PID 71203
Open:   http://localhost:8083/
Health: http://localhost:8083/health
Trace:  http://localhost:8083/hidden/prepack-info
```

ESTABLISH

The application starts successfully with the packed-and-installed dependency on port `8083`.

The PID is an observation from this run, not an invariant.

RUN

```
curl -sS http://localhost:8083/hidden/prepack-info | jq
```

OBSERVED OUTPUT

```
{
  "event": "npm-prepack-generated",
  "package": "trace-route-package",
  "version": "1.0.0",
  "generatedBy": "npm lifecycle prepack -> scripts/generate-dist.js",
  "message": "This runtime route was generated by an npm prepack lifecycle scr
  "origin": "trace-route-package build input",
  "route": "/hidden/prepack-info"
}
```


ESTABLISH

The generated package content is not merely present on disk. It changes runtime behaviour.

RUN

```
curl -sS http://localhost:8083/health | jq
```

OBSERVED OUTPUT


```
{
  "application": "node-prepack-trace-lab",
  "status": "UP"
}
```


ESTABLISH

Both forms of behaviour coexist in the running process:

```
/health
    source-defined application behaviour

/hidden/prepack-info
    dependency behaviour generated during npm prepack
```

---

1. **npm package creation can execute package-owned code before the distributable artefact is assembled.**
2. **The packed npm artefact can contain runtime files produced by that execution.**
3. **Publication rules can retain generated output while dropping the mechanism that produced it.**
4. **Package identity and build provenance are different evidence.** npm and Syft can identify `trace-route-package@1.0.0` in suitable context, but that identity does not explain how its runtime bytes came into existence.
5. **Scanner results depend on evidence context.** Syft found no package when scanning the isolated installed directory, but identified both npm packages when scanning the whole project.
6. **A dependency-derived SBOM can correctly identify the package and still omit its build transformation history.**
7. **Generated package content can directly affect runtime behaviour.** `/hidden/prepack-info` is served from content generated by the package lifecycle.

The complete evidence chain is:

```
package build input
        ↓
package-owned prepack script
        ↓
generated runtime files
        ↓
npm tarball
        ↓
installed node_modules package
        ↓
runtime endpoint
```

The evidence views are:

```
package source          → input + generator + generated output after prepack
tarball                 → generated output, no generator/input
installed package       → generated output, no generator/input
npm dependency tree     → package identity + relationship
npm CycloneDX SBOM      → package identity
Syft isolated package   → no package identified in this walkthrough
Syft whole project      → both npm package identities
runtime                 → generated behaviour executes
```

T01 KNOWN GROUND TRUTH — S04

T01 starts with facts already established by the completed S04 walkthrough.

SOFTWARE OF INTEREST

```
trace-injector-maven-plugin:1.0.0
        ↓
trace-route-payload:1.0.0
```

The payload contributes:

```
/hidden/build-info
```

The plugin converts that build-time input into generated Java source and ServiceLoader metadata.

KNOWN EVIDENCE VIEWS

Maven application dependency tree

S04 observed:

```
dev.noregressions.trace:maven-plugin-hidden-content:jar:1.0.0
```

There were no application dependencies beneath the root.

Maven plugin resolver

S04 observed:

```
trace-injector-maven-plugin:1.0.0
    trace-route-payload:1.0.0
```

Actual Maven plugin ClassRealm

S04 debug evidence included both:

```
Included: dev.noregressions.trace:trace-injector-maven-plugin:jar:1.0.0
Included: dev.noregressions.trace:trace-route-payload:jar:1.0.0
```

Final JAR

The final application JAR contains:

```
dev/noregressions/trace/s04/generated/GeneratedTraceRoute.class
META-INF/services/dev.noregressions.trace.s04.TraceRoute
META-INF/trace-lab/plugin-injection.properties
```

Maven-model CycloneDX

The completed S04 walkthrough observed:

```
CycloneDX: Creating BOM version 1.6 with 0 component(s)
```

Syft final-JAR scan

S04 observed one identified package:

```
maven-plugin-hidden-content  1.0.0  java-archive
```

Neither the plugin nor payload was recovered as package identity.

Runtime

```
{
  "message": "This runtime endpoint came from a transitive Maven plugin depend
  "origin": "trace-route-payload",
  "introducedBy": "trace-injector-maven-plugin",
  "route": "/hidden/build-info"
}
```

T01 SUCCESS CRITERION

T01 does **not** require Snyk to find more.

A useful result is any defensible answer to:

```
For each Snyk analysis mode:
  what evidence source did it use?
  what did it identify?
  did it expose plugin/payload identity?
  did it expose provenance?
  where did it stop?
```

S03 already established:

```
requirements.txt
    → reportkit 1.0.0 only

reportkit wheel METADATA
    → Requires-Dist: tracehook-demo==1.0.0

tracehook-demo 1.0.0
    → distributed as sdist
    → pyproject.toml nominates tracehook_backend
    → build backend runs during pip installation

original sdist
    → no tracehook_demo/__init__.py
    → no build-hook.json

generated wheel / installed environment
    → tracehook_demo/__init__.py present
    → build-hook.json present
```

```
runtime
    → reportkit imports generated tracehook_demo content
```

T01 asks whether Snyk preserves only the package dependency relationship or also the build-execution lineage that created the installed files.

```
source package → prepack declared + generator/input present
npm pack → prepack executes + dist generated
published tarball → package.json + dist only
installed package → publication output only
runtime → generated dist/index.js executes
```

T01 / S01 — SNYK AGAINST SPRING + NODE

This is the S01 case inside **T01 — Snyk Beyond the SBOM.**

S01 contains four useful tracer states:

```
jackson-databind 2.19.4
    ordinary Maven dependency
    survives as an intact nested JAR

commons-codec 1.17.1
    resolved by normalizer
    shaded and relocated to com.acme.internal.codec
    original Maven metadata survives inside normalizer

commons-codec 1.18.0
    selected in the service graph
    survives as an ordinary nested JAR

lodash 4.17.21
    ordinary npm dependency before the frontend build
    folded into Vite-generated JavaScript
    package boundary absent from deployable frontend
```

S01 already established an important controlled result:

```
shaded commons-codec bytecode + Maven metadata
    → Syft identifies commons-codec 1.17.1

same shaded bytecode - Maven metadata
    → Syft no longer identifies commons-codec

frontend/dist
    → Syft identifies 0 packages

final Spring Boot JAR
    → Syft identifies commons-codec 1.17.1
    → Syft identifies commons-codec 1.18.0
    → Syft identifies jackson-databind 2.19.4
    → no lodash package identified
```

The Snyk questions are:

> Does Snyk behave like the Maven model, like Syft's artefact inspection, or differently at each evidence boundary?

and:

> Can Snyk recover package identity after shading or JavaScript bundling?

RUN

```
./scripts/baseline-s01.sh
```

The baseline also populates an isolated Maven repository under:

```
results/s01/maven-repo
```

because Snyk's Maven provenance mode requires resolved artefacts to be present in a local Maven repository.

OBSERVE

Capture during walkthrough.

ESTABLISH

Confirm:

```
normalizer resolves commons-codec 1.17.1
service graph resolves commons-codec 1.18.0
service resolves jackson-databind 2.19.4
npm resolves lodash 4.17.21

normalizer contains relocated com.acme.internal.codec bytecode
normalizer retains commons-codec Maven metadata
metadata-stripped normalizer retains bytecode but loses that metadata

frontend/dist contains generated assets, not node_modules/lodash

service JAR contains:
    jackson-databind 2.19.4
    normalizer 1.0.0
    commons-codec 1.18.0
    frontend static assets
```

RUN

```
./scripts/run-snyk-s01.sh
```

The first four probes are:

```
Snyk Maven aggregate test
Snyk Maven aggregate test + --include-provenance
Snyk Maven aggregate CycloneDX
Snyk Maven aggregate CycloneDX + --include-provenance
```

QUESTION

Does the Maven reactor view expose both commons-codec versions because it sees both module contexts?

Does it see the physically embedded shaded `1.17.1` as shipped software, or merely as a dependency of the normalizer project?

OBSERVE

Capture during walkthrough.

ESTABLISH

Keep project-model knowledge separate from physical-inclusion evidence.

QUESTION

Before Vite bundling, can Snyk identify:

```
lodash 4.17.21
react 18.3.1
react-dom 18.3.1
```

from the npm project and lockfile?

OBSERVE

Capture during walkthrough.

ESTABLISH

This is the strongest package-identity point for lodash.

QUESTION

Once only Vite output remains, can Snyk still identify lodash?

OBSERVE

Capture during walkthrough.

ESTABLISH

Compare with the npm-source result.

The important distinction is:

```
build workspace package identity
        !=
deployable bundle package identity
```

QUESTION

Syft identified `commons-codec 1.17.1` from the shaded JAR because Maven package metadata survived.

Can Snyk's unmanaged JAR mode do the same?

OBSERVE

Capture during walkthrough.

ESTABLISH

This is a direct comparison between two artefact scanners over exactly the same custom JAR.

QUESTION

Does the controlled evidence-loss experiment change Snyk's result?

```
normalizer-1.0.0.jar
    relocated codec bytecode
    + commons-codec Maven metadata

normalizer-no-codec-metadata.jar
    same relocated codec bytecode
    - commons-codec Maven metadata
```

OBSERVE

Capture during walkthrough.

ESTABLISH

If Snyk gives the same answer for both, then its unmanaged identification mechanism differs materially from Syft's.

If the answer changes, record exactly which evidence it used.

QUESTION

Can direct unmanaged analysis of the custom executable JAR identify anything useful?

Then, after unpacking it, can recursive unmanaged scanning identify intact nested JARs such as:

```
jackson-databind 2.19.4
commons-codec 1.18.0
```

Can it recover `commons-codec 1.17.1` inside the nested shaded normalizer?

Can it recover lodash from the packaged static frontend bundle?

OBSERVE

Capture during walkthrough.

ESTABLISH

Compare:

```
custom top-level application archive
intact nested dependency JARs
transformed shaded bytes
bundled JavaScript bytes
```

RUN

```
./scripts/compare-s01.sh
```

Complete this matrix from actual output:

Evidence view	Jackson 2.19.4	codec 1.17.1 shaded	codec 1.18.0	lodash 4.17.21
Maven module resolution	?	?	?	n/a
npm source/lockfile	n/a	n/a	n/a	?
Maven CycloneDX	?	?	?	no
Syft normalizer	n/a	yes	n/a	n/a
Syft stripped normalizer	n/a	no	n/a	n/a
Syft frontend/dist	n/a	n/a	n/a	no
Syft final service JAR	yes	yes	yes	no
Snyk Maven aggregate	?	?	?	n/a
Snyk Maven + provenance	?	?	?	n/a
Snyk npm source	n/a	n/a	n/a	?
Snyk frontend/dist	n/a	n/a	n/a	?
Snyk shaded normalizer	n/a	?	n/a	n/a
Snyk stripped normalizer	n/a	?	n/a	n/a
Snyk final service JAR	?	?	?	?
Snyk unpacked service	?	?	?	?

The central S01/T01 question is:

> Which kinds of software identity survive a transformation strongly enough for Snyk to recover them, and which require evidence from an earlier supply-chain boundary?

T01 / S02 — SNYK AGAINST PAYARA + MVNPM

This is the second case inside **T01 — Snyk Beyond the SBOM.**

S02 gives us a different kind of missing software from S04.

The tracer is:

```
org.mvnpm:lodash-es:4.17.21
```

S02 already established:

```
application project dependency tree
        → no lodash-es

Maven plugin execution realm
        → lodash-es present

esbuild source map
        → lodash-es modules contributed to browser bundle

generated-web Syft scan
        → 0 packages

WAR
        → app.js + app.js.map ship
        → lodash-es package identity absent

WAR Syft scan
        → application + commons-lang3
        → no lodash-es

Maven CycloneDX
        → project-model components
        → no lodash-es
```

T01 now asks:

> Can any Snyk analysis mode recover `lodash-es` after it has moved from Maven plugin dependency identity into bundled JavaScript?

There is also a positive control:

```
commons-lang3
```

Unlike `lodash-es`, its original Maven JAR survives inside `WEB-INF/lib`. Snyk's unmanaged artefact scanning should therefore have much better evidence to work with.

RUN

```
./scripts/baseline-s02.sh
```

OBSERVED OUTPUT

Capture during walkthrough.

ESTABLISH

Confirm:

```
lodash-es
    absent from application dependency tree
    present in actual Maven plugin execution realm
    visible in source-map evidence
    absent as identified package from generated-web/WAR Syft views
```

Also record the treatment of:

```
commons-lang3
Jakarta provided dependencies
```

RUN

```
./scripts/run-snyk-s02.sh
```

The harness performs:

```
1. normal Snyk Maven test
2. Maven test + --include-provenance
3. Snyk CycloneDX
4. Snyk CycloneDX + --include-provenance
5. unmanaged scan of final WAR
6. unmanaged scan of embedded commons-lang3 JAR
7. recursive unmanaged scan of the unpacked WAR
```

OBSERVED OUTPUT

Capture during walkthrough.

QUESTION

Does Snyk identify:

```
commons-lang3?
Jakarta provided dependencies?
lodash-es?
esbuild plugin dependencies?
```

OBSERVED OUTPUT

Capture during walkthrough.

ESTABLISH

Do not assume Snyk's Maven view is identical to either `dependency:tree` or the Maven CycloneDX output. Record exactly what it emits.

QUESTION

Does Snyk's own SBOM recover anything missing from the Maven-generated CycloneDX?

Particularly:

```
lodash-es
```

OBSERVED OUTPUT

Capture during walkthrough.

ESTABLISH

Compare component identity, not just SBOM format.

QUESTION

Does `--include-provenance`:

```
add fingerprints?
add PURLs?
add components?
recover lodash-es?
```

OBSERVED OUTPUT

Capture during walkthrough.

ESTABLISH

Distinguish **identity enrichment** from **broader component discovery**, as T01/S04 already required.

WHY

The project model may have lost information that remains in the final bytes.

QUESTION

Can Snyk identify:

```
commons-lang3 from the WAR?
lodash-es from bundled JavaScript?
the application WAR itself?
```

OBSERVED OUTPUT

Capture during walkthrough.

ESTABLISH

Record whether Snyk can inspect the custom WAR directly and what it can name.

Snyk's current documentation supports individual JAR/WAR/AAR unmanaged scans, while noting limitations for custom-built artefacts.

WHY

We need to know whether a failure to identify `lodash-es` is simply because unmanaged scanning is not working.

The WAR contains the original `commons-lang3` JAR, whose identity should be recoverable from published Maven artefact evidence.

OBSERVED OUTPUT

Capture during walkthrough.

ESTABLISH

If Snyk identifies `commons-lang3` here, we have a strong contrast:

```
original dependency JAR survives
        → package identity recoverable

dependency transformed into JS bundle
        → test whether package identity remains recoverable
```

WHY

Snyk's documentation recommends extracting container archives when you want it to inspect embedded dependency JARs.

OBSERVED OUTPUT

Capture during walkthrough.

ESTABLISH

Compare direct WAR inspection with access to the embedded JARs as individual artefacts.

RUN

```
./scripts/compare-s02.sh
```

OBSERVED OUTPUT

Capture during walkthrough.

ESTABLISH

Complete this matrix:

Evidence view	commons-lang3	Jakarta API	lodash-es	Build/bundle provenance
Maven dependency tree	?	?	?	?
Maven plugin ClassRealm	?	?	?	?
Maven CycloneDX	?	?	?	?
Syft generated-web	?	?	?	?
Syft WAR	?	?	?	?
Snyk Maven test	?	?	?	?
Snyk Maven + provenance	?	?	?	?
Snyk CycloneDX	?	?	?	?
Snyk final WAR	?	?	?	?
Snyk embedded JAR	?	n/a	n/a	?
Snyk unpacked WAR	?	?	?	?

The central T01/S02 question is:

> Can a scanner recover package identity after a build transformation has kept the code but discarded the original package boundary?

T01 / S03 — SNYK AGAINST PYTHON + PEP 517

This is the S03 case inside **T01 — Snyk Beyond the SBOM**.

S03 is different from S01, S02 and S04.

The interesting fact is not merely that a component was transformed. A **transitive dependency executes build code during installation**.

Known S03 ground truth is:

```
requirements.txt
    reportkit==1.0.0

        ↓ wheel metadata

Requires-Dist:
    tracehook-demo==1.0.0

        ↓ local repository

tracehook_demo-1.0.0.tar.gz
    pyproject.toml
    tracehook_backend.py

        ↓ PEP 517

tracehook_backend.build_wheel()

        ↓ creates files absent from sdist

tracehook_demo/__init__.py
tracehook_demo/build-hook.json

        ↓ generated wheel

site-packages

        ↓ runtime

reportkit.runtime_trace()
```

The central T01/S03 question is:

> Can Snyk identify the transitive Python package while also preserving the fact that dependency-supplied build code executed and generated new runtime content?

Those are two different kinds of knowledge:

```
package inventory
        !=
build execution history
```

Snyk's current Python CLI model uses supported Python manifests such as `requirements.txt` and the installed Python environment to construct the dependency graph. Snyk's experimental `--include-provenance` option is Maven-specific, so it is deliberately **not** used for this Python case.

RUN

```
./scripts/baseline-s03.sh
```

OBSERVE

Capture during walkthrough.

ESTABLISH

Confirm all of the following:

```
application declaration
    reportkit==1.0.0

reportkit wheel metadata
    Requires-Dist: tracehook-demo==1.0.0

tracehook-demo distribution
    sdist

sdist build-system
    build-backend = "tracehook_backend"

sdist files
    tracehook_demo/__init__.py absent
    tracehook_demo/build-hook.json absent

pip install
    builds a wheel for tracehook-demo

installed environment
    reportkit 1.0.0
    tracehook-demo 1.0.0

installed files
    tracehook_demo/__init__.py present
    tracehook_demo/build-hook.json present

runtime
    generated marker affects application behaviour
```

The baseline also performs a second wheel build and retains the generated wheel so we can inspect the post-build artefact directly.

RUN

```
./scripts/run-snyk-s03.sh
```

The first probe runs:

```
snyk test
    requirements.txt
```

```
    +
    the S03 virtual environment
```

QUESTION

Can Snyk recover:

```
reportkit 1.0.0
    ↓
tracehook-demo 1.0.0
```

even though `requirements.txt` names only `reportkit`?

OBSERVE

Capture during walkthrough.

ESTABLISH

If it does, Snyk is recovering more than the literal top-level requirements file because the installed Python environment supplies the transitive dependency information.

That still does not tell us whether Snyk knows **how** `tracehook-demo` was built.

QUESTION

Does the Snyk SBOM contain:

```
reportkit
tracehook-demo
reportkit → tracehook-demo
```

OBSERVE

Capture during walkthrough.

ESTABLISH

Separate these questions:

```
Does the SBOM know the package exists?
Does the SBOM know the dependency relationship?
Does the SBOM know the distribution was an sdist?
Does the SBOM know a build backend executed?
```

Do not infer the latter two from the first two.

The harness explicitly searches Snyk text and JSON output for:

```
tracehook_backend
build_wheel
build-hook.json
pep517-build-backend-executed
```

WHY

These strings are directly present in S03's actual build/runtime evidence.

If Snyk reconstructs the build execution path, we should be able to point to where that fact appears.

OBSERVE

Capture during walkthrough.

ESTABLISH

Absence here does not mean the build hook did not execute. S03 already proved that independently.

It means that execution fact is not represented in the Snyk evidence we tested.

INPUT

```
tracehook_demo-1.0.0.tar.gz
    ↓ unpacked
pyproject.toml
tracehook_backend.py
```

QUESTION

Does Snyk Open Source treat this generic PEP 517/PEP 621 source tree as a supported Pip project on its own?

OBSERVE

Capture during walkthrough.

ESTABLISH

This tests whether Snyk can move backwards from the dependency's source distribution into its build configuration without a `requirements.txt`-style application manifest.

The generated wheel contains the package identity and the generated files:

```
tracehook_demo/__init__.py
tracehook_demo/build-hook.json
tracehook_demo-1.0.0.dist-info/METADATA
```

QUESTION

Can normal Snyk Open Source project discovery use that artefact directly?

OBSERVE

Capture during walkthrough.

ESTABLISH

This distinguishes:

```
Python package artefact metadata
        from
Snyk-supported project manifest
```

QUESTION

If only the installed environment remains, without the application's `requirements.txt`, does Snyk discover a project from `.dist-info` package metadata?

OBSERVE

Capture during walkthrough.

ESTABLISH

This tells us whether Snyk's Python dependency analysis is fundamentally anchored on the application/project manifest even though it consults the installed environment to build the complete dependency graph.

RUN

```
./scripts/compare-s03.sh
```

Complete this matrix from actual output:

Evidence view	reportkit	tracehook-demo	sdist fact	backend execution	generated files
requirements.txt	yes	no	no	no	no
reportkit wheel metadata	yes	yes	no	no	no
tracehook sdist	n/a	yes	yes	backend declared	files absent
pip build log	yes	yes	yes	build observed	wheel created
installed environment	yes	yes	history lost	history lost	files present
runtime marker	indirect	yes	says generated	names build_wheel	yes
Snyk Pip test	?	?	?	?	?
Snyk Python SBOM	?	?	?	?	?
Snyk sdist discovery	n/a	?	input is sdist tree	?	absent
Snyk wheel discovery	n/a	?	history not intrinsic	?	present
Snyk site-packages discovery	?	?	history not intrinsic	?	present

The central result we are testing is:

```
A scanner may reconstruct package dependency identity
without reconstructing the executable build steps that produced
```

```
the installed package contents.
```

T01 / S05 — SNYK AGAINST NPM PREPACK

This is the final scenario case inside **T01 — Snyk Beyond the SBOM.**

```
trace-route-package source
    package.json
    build-input/route.json
    scripts/generate-dist.js
        ↓ npm pack / prepack
    dist/index.js
    dist/prepack-evidence.json
        ↓ files=[dist]
published tarball
    package.json + dist
    generator absent
    build input absent
        ↓ npm install
node_modules/trace-route-package
```

The question is: **Can Snyk identify the installed npm package while preserving the publication-time fact that `prepack` executed and manufactured the runtime files?**

`package inventory != publication history`

RUN

```
./scripts/baseline-s05.sh
./scripts/run-snyk-s05.sh
./scripts/compare-s05.sh
```

The harness compares source, npm-pack evidence, packed tarball, installed package, npm SBOM, Snyk application dependency analysis, Snyk CycloneDX, and direct scans of source/published/installed package forms.

T01 — SNYK BEYOND THE SBOM

QUESTION

> What does a commercial SCA tool know that an ordinary SBOM does not, and which supply-chain transformations remain invisible even to it?

This investigation reuses five completed trace labs:

```
S01  Spring + Node
S02  Payara + mvnpm
S03  Python + PEP 517
S04  Maven plugin hidden-content
S05  Node + npm prepack
```

The investigation compares several evidence types:

```
source declarations
package-manager dependency models
build-tool execution evidence
generated artefacts
SBOMs
Snyk project scans
Snyk artefact/unmanaged scans
runtime evidence
```

The central result is:

```
better package identity
        !=
complete supply-chain history
```

Snyk can often provide richer identity, vulnerability and dependency-path information than a basic SBOM. But it does not reconstruct every build realm, lifecycle action, code-generation step, bundling transformation, or publication-time mutation after those facts have disappeared from the dependency model.

GROUND TRUTH

The lab contains four useful tracer states:

```
jackson-databind 2.19.4
    ordinary Maven dependency
    physically shipped as intact nested JAR

commons-codec 1.17.1
    dependency of normalizer
    shaded and relocated into normalizer
    Maven package metadata survives

commons-codec 1.18.0
    selected by service dependency management
    physically shipped as intact nested JAR
```

```
lodash 4.17.21
    npm dependency
    bundled by Vite into generated browser JavaScript
    package boundary disappears
```

Maven resolves:

```
normalizer
    → commons-codec 1.17.1

service
    → normalizer
        → commons-codec 1.18.0
    → jackson-databind 2.19.4
```

The final Spring Boot JAR physically contains:

```
BOOT-INF/lib/jackson-databind-2.19.4.jar
BOOT-INF/lib/normalizer-1.0.0.jar
BOOT-INF/lib/commons-codec-1.18.0.jar
```

The normalizer JAR also contains relocated `commons-codec 1.17.1` bytecode.

Syft can identify `commons-codec 1.17.1` inside the shaded normalizer while its Maven metadata survives. Removing only:

```
META-INF/maven/commons-codec/commons-codec/
```

makes that identity disappear from Syft while leaving the relocated bytecode unchanged.

SNYK MAVEN PROJECT RESULT

Snyk's Maven aggregate view identifies both codec versions in their respective module contexts:

```
normalizer
    → commons-codec 1.17.1

service
    → normalizer
        → commons-codec 1.18.0
    → jackson-databind 2.19.4
```

This proves Snyk knows both versions from the Maven reactor model.

It does **not** prove that Snyk independently reconstructed the physical fact that both versions' code ship in the final application.

SNYK PROVENANCE RESULT

`--include-provenance` preserves the same component set.

For external Maven artefacts it adds checksum-qualified PURLs such as:

```
pkg:maven/commons-codec/commons-codec@1.17.1?checksum=sha1:...
pkg:maven/commons-codec/commons-codec@1.18.0?checksum=sha1:...
```

```
pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.19.4?checksum=sha1:...
```

The root reactor POM gains:

```
?type=pom
```

The result is:

> Snyk provenance strengthens the identity of software already present in its dependency model. It does not broaden the dependency model.

SNYK NPM RESULT

From the npm project Snyk identifies:

```
lodash 4.17.21
```

and reports vulnerability information for it.

After Vite bundling, scanning `frontend/dist` produces:

```
No supported files found
```

So the correct interpretation is not:

```
Snyk inspected the bundle and failed to recognise lodash
```

but:

```
the deployable bundle no longer presents a supported npm project boundary
```

No tested artefact-oriented Snyk view recovered `lodash 4.17.21` from the shipped browser bundle.

SNYK UNMANAGED JAR RESULT

Direct unmanaged scans of both:

```
normalizer-1.0.0.jar
normalizer-no-codec-metadata.jar
```

produce an `unknown` custom JAR.

Unlike Syft, Snyk unmanaged scanning did not identify shaded `commons-codec 1.17.1` from the surviving Maven metadata.

Scanning the complete Spring Boot JAR also produced one unknown custom JAR.

After unpacking the Spring Boot JAR, recursive unmanaged scanning identifies intact nested libraries such as:

```
commons-codec 1.18.0
jackson-databind 2.19.4
```

while the custom shaded normalizer remains unknown.

SO1 ESTABLISHES

```
known from model + shipped intact
    → recoverable from Maven model
    → recoverable again when intact nested package boundary survives

known from model + transformed
    → visible before transformation
    → may become unrecoverable afterwards

known from npm model + bundled
    → visible before bundling
    → package identity can disappear from deployable output
```

GROUND TRUTH

The application has:

```
commons-lang3 3.18.0
    ordinary application dependency
    physically shipped in WEB-INF/lib

Jakarta EE Web API 11.0.0
    Maven provided dependency
    present in application dependency model
    absent from WAR

lodash-es 4.17.21
    dependency of esbuild Maven plugin
    present in actual Maven plugin ClassRealm
    contributes source modules to generated browser bundle
    absent from application dependency graph
```

The generated source map proves `lodash-es` source modules contributed to the browser bundle.

SNYK PROJECT RESULT

Snyk's Maven project scan identifies the application dependency graph, including:

```
Jakarta provided dependencies
commons-lang3 3.18.0
```

It does not identify:

```
lodash-es 4.17.21
```

because that package exists in the Maven plugin realm rather than the application dependency graph.

PROVENANCE

Normal `snyk test` JSON emitted no PURLs.

With `--include-provenance`, Snyk added PURLs for the same Maven dependency set:

```
root
    → ?type=war

dependencies
    → ?checksum=sha1:...
```

The CycloneDX comparison showed the same component set before and after provenance enrichment.

Critically, Snyk checksum-qualified Jakarta `provided` dependencies even though they were not physically shipped in the WAR.

Therefore:

```
provenance identity
        !=
physical inclusion proof
```

ARTEFACT SCANNING

Direct unmanaged scan of the whole WAR:

```
unknown custom WAR
```

After unpacking the WAR, Snyk identifies preserved `commons-lang3 3.18.0`.

It does not reconstruct `lodash-es 4.17.21` from the generated browser bundle.

SO2 ESTABLISHES

```
modelled but not shipped
    → Jakarta provided APIs

modelled and shipped intact
    → commons-lang3

not modelled by the application but code ships
    → lodash-es
```

Snyk's project/provenance model handles the first two well, but the Maven plugin-domain contribution remains invisible.

GROUND TRUTH

The application declares:

```
reportkit==1.0.0
```

The `reportkit` wheel metadata declares:

```
Requires-Dist: tracehook-demo==1.0.0
```

`tracehook-demo` is supplied as an sdist containing:

```
pyproject.toml
tracehook_backend.py
```

Its PEP 517 configuration declares:

```
build-backend = "tracehook_backend"
backend-path = ["."]
```

The original sdist does not contain:

```
tracehook_demo/__init__.py
tracehook_demo/build-hook.json
```

During `pip install`, pip builds a wheel and executes:

```
tracehook_backend.build_wheel()
```

The generated wheel contains both runtime files.

The installed marker records:

```
event       = pep517-build-backend-executed
generatedBy = tracehook_backend.build_wheel
```

and the generated content affects runtime behaviour.

SNYK PIP RESULT

Snyk reconstructs the full installed dependency graph:

```
S03-python-pep517
    → reportkit 1.0.0
        → tracehook-demo 1.0.0
```

Its CycloneDX SBOM preserves:

```
application
    → reportkit
        → tracehook-demo
```

MISSING EXECUTION HISTORY

The actual Snyk outputs contain no evidence for:

```
tracehook_backend
build_wheel
build-hook.json
pep517-build-backend-executed
```

Snyk can therefore reconstruct package dependency identity without reconstructing the executable PEP 517 build history that produced the installed files.

PACKAGE ARTEFACT BOUNDARIES

In the tested modes, Snyk Open Source did not treat any of these as a supported project on their own:

```
unpacked tracehook sdist
unpacked generated wheel
installed site-packages directory
```

The successful scan is anchored on the application `requirements.txt` plus the installed Python environment.

S03 ESTABLISHES

```
dependency graph knowledge
        !=
build execution history
```

GROUND TRUTH

The application has no ordinary third-party runtime dependencies.

During the build:

```
trace-injector-maven-plugin
    → trace-route-payload
```

exist in the Maven plugin resolution domain and actual plugin ClassRealm.

The plugin generates:

```
GeneratedTraceRoute.class
META-INF/services/dev.noregressions.trace.s04.TraceRoute
META-INF/trace-lab/plugin-injection.properties
```

The resulting application exposes:

```
/hidden/build-info
```

even though that route did not exist in checked-in application source.

APPLICATION DEPENDENCY VIEWS

Maven application dependency tree:

```
application only
```

Maven CycloneDX:

```
0 dependency components
```

Syft final-JAR scan:

```
application archive only
```

SNYK PROJECT RESULT

Normal Snyk Maven analysis identifies:

```
application only
```

It does not identify:

```
trace-injector-maven-plugin
trace-route-payload
```

SNYK PROVENANCE

`--include-provenance` adds a Maven PURL to the known root application artefact.

It does not discover the plugin or payload.

In Snyk's CycloneDX output, provenance produced no substantive component-set change beyond run-specific metadata.

UNMANAGED JAR SCAN

The final custom JAR is reported as:

```
unknown
```

The scan exposes uncertainty but does not reconstruct the plugin/payload relationship or build history.

S04 ESTABLISHES

```
Maven dependency tree     → application only
Maven plugin resolver     → plugin + payload
Maven plugin ClassRealm   → plugin + payload
Maven CycloneDX           → no dependency components
Snyk Maven                → application only
Snyk + provenance         → same known application identity, enriched
Snyk unmanaged JAR        → unknown custom JAR
runtime                   → plugin-generated behaviour executes
```

The build-tool dependency realm is a distinct evidence source.

GROUND TRUTH

The application depends on:

```
trace-route-package 1.0.0
```

The package source declares:

```
{
  "main": "dist/index.js",
  "files": ["dist"],
  "scripts": {
```

```
    "prepack": "node scripts/generate-dist.js"
  }
}
```

The source contains:

```
build-input/route.json
scripts/generate-dist.js
```

The clean build removes any old `dist/`, then `npm pack` records:

```
trace-route-package@1.0.0 prepack
node scripts/generate-dist.js
generated dist/index.js
generated dist/prepack-evidence.json
```

The resulting tarball contains only:

```
package.json
dist/index.js
dist/prepack-evidence.json
```

It does **not** contain:

```
scripts/generate-dist.js
build-input/route.json
```

The installed package preserves the same publication boundary.

Runtime imports the generated `dist/index.js`.

A PARTICULARLY AWKWARD PUBLICATION STATE

The published `package.json` still declares:

```
prepack = node scripts/generate-dist.js
```

but the referenced file:

```
scripts/generate-dist.js
```

is absent from the published package.

Therefore the published package contains a lifecycle declaration but not the executable source that previously satisfied it.

That declaration alone is not proof that the lifecycle hook executed.

The `npm pack` log provides that proof.

SNYK APPLICATION RESULT

Snyk identifies:

```
node-prepack-trace-lab 1.0.0
    → trace-route-package 1.0.0
```

The Snyk CycloneDX SBOM preserves the same relationship.

SNYK PACKAGE SCANS

Snyk can independently scan:

```
source trace-route-package
unpacked published tarball
installed trace-route-package
```

because each retains `package.json`.

All scans succeed.

MISSING PUBLICATION HISTORY

The real Snyk outputs contain no evidence for:

```
scripts/generate-dist.js
npm-prepack-generated
prepack-evidence.json
/hidden/prepack-info
```

Textual matches for `prepack` are only incidental occurrences in project/path names such as:

```
node-prepack-trace-lab
```

They are not lifecycle-script evidence.

Even when scanning the source package, where the lifecycle declaration and generator physically exist, the Snyk Open Source result does not expose the `prepack` action as supply-chain execution evidence.

S05 ESTABLISHES

```
npm package identity
        !=
npm publication history
```

Snyk knows what package is present, but the tested dependency views do not explain how npm manufactured the files that package published.

Evidence / capability	S01	S02	S03	S04	S05
Normal dependency model	Java + npm identities visible before transformation	App Maven deps visible	Python transitive package recovered	Plugin/payload absent	npm package recovered
Snyk SBOM	Same project identities	Same app Maven identities	Preserves Python transitive edge	Plugin/payload absent	Preserves npm dependency edge
Provenance enrichment	Same Maven set + checksum PURLs	Same Maven set + checksum PURLs	not applicable in tested Python mode	Root identity enriched only	not used

Evidence / capability	S01	S02	S03	S04	S05
Build-time dependency realm	not reconstructed from final artefact	lodash-es plugin realm missed	PEP 517 backend execution not represented	plugin + payload missed	lifecycle execution not represented
Generated/bundled content lineage	lodash not recovered after bundle	lodash-es not recovered after bundle	generated Python files not tied to backend	generated route not tied to plugin	generated dist not tied to prepack
Intact package artefact recovery	nested JARs recoverable after unpack	commons JAR recoverable after unpack	project anchored to manifest + environment	custom JAR unknown	package.jso remains enough for npm project scan
Transformed custom artefact recovery	shaded normalizer unknown to Snyk unmanaged	n/a	n/a	custom application JAR unknown	n/a
Evidence outside Snyk still required	shade metadata / physical JAR / bundling evidence	Maven plugin ClassRealm + source map	pip build log + sdist/backend + generated marker	Maven plugin resolver/ClassRealm	npm pack log + generator + generated evidence

---

1. SNYK CAN KNOW MORE THAN A LITERAL SOURCE DECLARATION

Examples:

```
requirements.txt
    reportkit only

Snyk
    reportkit → tracehook-demo
```

and:

```
npm application
    → trace-route-package
```

Snyk dependency analysis reconstructs dependency relationships rather than merely echoing top-level declarations.

2. SNYK CAN PROVIDE MATERIALLY RICHER SECURITY INTELLIGENCE THAN A BASIC INVENTORY

The investigation observed Snyk adding:

```
dependency paths
vulnerability intelligence
fix guidance
```

```
PURLs
checksum-qualified Maven identities in provenance mode
```

That is useful information beyond a minimal component list.

3. PROVENANCE ENRICHMENT IS NOT PROVENANCE RECONSTRUCTION

For the Maven cases:

```
--include-provenance
    → stronger identities for known Maven artefacts
```

It did not discover:

```
Maven plugin-realm packages
code-generation history
bundling history
lifecycle execution
```

Therefore:

```
provenance enrichment
        !=
supply-chain history reconstruction
```

4. PACKAGE-MANAGER MODELS AND SHIPPED ARTEFACTS ANSWER DIFFERENT QUESTIONS

A model may include something not shipped:

```
Jakarta provided dependencies
```

A shipped artefact may contain something absent from the application model:

```
lodash-es code introduced through a Maven plugin
```

A transformed artefact may contain code whose package identity has become difficult or impossible for the tested scanner to recover:

```
bundled lodash
shaded custom JAR contents
```

5. BUILD SYSTEMS HAVE DEPENDENCY AND EXECUTION DOMAINS OUTSIDE THE APPLICATION DEPENDENCY GRAPH

Observed examples:

```
Maven plugin ClassRealm
PEP 517 build backend
```

```
npm prepack lifecycle
```

These domains can execute code and materially change the resulting application without becoming ordinary runtime dependency edges.

6. GENERATED CONTENT DOES NOT CARRY ITS OWN HISTORY AUTOMATICALLY

Observed examples:

```
Vite browser bundle
PEP 517 generated Python files
Maven-plugin-generated Java/service metadata
npm-prepack-generated dist files
```

A scanner observing only the final output may identify some surviving package boundaries, but it cannot be assumed to reconstruct the process that created those bytes.

7. EVIDENCE MUST BE COLLECTED AT THE BOUNDARY WHERE THE FACT STILL EXISTS

For these labs:

```
Maven application graph
    → ordinary application dependency resolution

Maven plugin ClassRealm
    → build-plugin dependencies

npm/package-lock
    → Node package identity before bundling

pip build log + sdist
    → PEP 517 build execution

npm pack log
    → publication lifecycle execution

final artefact scan
    → software whose package boundaries survive strongly enough
```

No single one of these views is the complete supply-chain record.

The strongest T01 conclusion is not that Snyk is deficient.

It is that software supply-chain evidence is **boundary-specific.**

Snyk is good at the dependency models and package identities it supports, and can enrich those identities with useful security and provenance information.

But once a build or publication transformation crosses out of that model:

```
plugin execution
shading
bundling
PEP 517 build hooks
```

```
npm lifecycle scripts
generated code
```

the missing history cannot be assumed to be recoverable later from the final artefact.

The practical rule is:

> Capture supply-chain evidence when the transformation happens. Do not rely on a later scanner to reconstruct facts that the build has already discarded.

T02 — DOCKER SCOUT: WHAT DOES THE FINAL CONTAINER KNOW?

OBJECTIVE

Use Docker Scout as a final-container evidence source for the workshop scenarios that produce container images.

Current coverage:

```
S01  Spring + Node
S02  Payara + mvnpm
```

The investigation asks:

> Once dependency resolution, plugin execution, shading, bundling, packaging and container assembly are complete, what software identity can Docker Scout recover from the final image, and what history has already been lost?

The tested Scout views are:

```
docker scout quickview
docker scout sbom --format list
docker scout cves
docker scout recommendations
```

All scans use:

```
local://IMAGE
```

so Scout analyses the exact locally built scenario image.

LOOK

Important tracer states:

```
jackson-databind 2.19.4
    ordinary Maven dependency
    shipped as intact nested JAR

commons-codec 1.18.0
    service-selected Maven dependency
    shipped as intact nested JAR

commons-codec 1.17.1
    dependency of normalizer
    shaded and relocated
    Maven package identity metadata survives

normalizer 1.0.0
    custom application library

lodash 4.17.21
    npm dependency
```

```
    bundled by Vite into browser JavaScript
    npm package boundary disappears
```

RUN

```
./scripts/baseline-s01.sh
./scripts/run-scout-s01.sh
./scripts/compare-s01.sh
```

OBSERVE

The rebuilt image was:

```
registry.example.com/checkout-service:release-123
sha256:7d3b4c23011efff02feddce24a181f921f5d9a46fb9b8a72c940d316fe7dae3c
```

The Docker build exported an attestation manifest.

Syft, scanning the same final image, catalogued:

```
179 packages
837 executables
```

and identified:

```
commons-codec      1.17.1
commons-codec      1.18.0
jackson-databind   2.19.4
normalizer         1.0.0
```

Docker Scout reported:

```
237 packages indexed
base image: eclipse-temurin:21-jre-jammy
provenance obtained from attestation
policy: FAILED (3/7)
health score: D (44%)
```

Scout identified the same four Java tracers:

```
commons-codec      1.17.1
commons-codec      1.18.0
jackson-databind   2.19.4
normalizer         1.0.0
```

Scout did not identify:

```
lodash 4.17.21
```

The tracer CVE view reported five vulnerabilities, all on:

```
jackson-databind 2.19.4
    2 HIGH
```

```
    3 MEDIUM
```

Scout's provenance view linked the image to:

```
https://github.com/noregressions/kcdc2026workshop.git
commit c8c745d5a7d4e8b83813965de87b7ac167d75db9
```

The base-image recommendation view said the current:

```
eclipse-temurin:21-jre-jammy
```

was up to date, while also suggesting Java 25 and Java 26 tags as alternative major-runtime upgrades with one fewer vulnerability in the base-image comparison.

ESTABLISH

```
Maven/npm model             → application dependency identity before transform
Syft final image            → 179 packages
Docker Scout final image    → 237 packages
```

Both final-image scanners recover the two codec versions, Jackson and normalizer.

Neither recovers lodash after Vite bundling destroys the npm package boundary.

Moving the scanner later expands the deployed software universe, but does not guarantee recovery of identities destroyed earlier.

LOOK

Important tracer states:

```
commons-lang3 3.18.0
    ordinary application dependency
    shipped intact in WAR

Jakarta EE Web API 11.0.0
    Maven provided dependency
    present in application model
    absent from WAR

lodash-es 4.17.21
    Maven plugin dependency
    present in esbuild ClassRealm
    source-map evidence proves code entered browser bundle
    package identity lost after bundling

Payara server
    supplies its own Jakarta/API/runtime package universe
```

RUN

```
./scripts/baseline-s02.sh
./scripts/run-scout-s02.sh
./scripts/compare-s02.sh
```

OBSERVE

The rebuilt image was:

```
payara-mvnpm-trace-lab:local
sha256:9ad9bb4ab46971e6639855c48b7769864c437cf43ab7e786595613895f40a98c
```

The Docker build exported an attestation manifest.

Syft catalogued:

```
589 packages
825 executables
```

It identified:

```
commons-lang3 3.18.0
payara-mvnpm-trace-lab 1.0.0
many Jakarta API packages supplied by Payara
```

It did not identify:

```
lodash-es 4.17.21
```

Docker Scout reported:

```
655 packages indexed
base image: payara/server-web:7.2026.7
provenance obtained from attestation
policy: FAILED (3/7)
health score: D (44%)
```

Scout identified:

```
commons-lang3 3.18.0
many Jakarta APIs
many Payara/GlassFish modules
payara-mvnpm-trace-lab 1.0.0
```

Scout did not identify:

```
lodash-es 4.17.21
```

The filtered CVE probe used package-name patterns for:

```
commons-lang3
lodash-es
payara
jakarta
```

Those patterns matched a large set of Payara/Jakarta-named packages. Scout reported:

```
No vulnerable packages detected
```

for that filtered set.

Scout's provenance view linked the image to:

```
https://github.com/noregressions/kcdc2026workshop.git
commit c8c745d5a7d4e8b83813965de87b7ac167d75db9
```

Scout reported:

```
No recommendations
```

for the `payara/server-web:7.2026.7` base image.

ESTABLISH

The Jakarta APIs demonstrate a particularly important boundary change:

```
Maven application model
    → Jakarta APIs present as provided dependencies

WAR
    → Jakarta APIs absent

Payara container image
    → Jakarta APIs present again
       because the server supplies them
```

The container scan proves that Jakarta software is deployed with the application.

It does **not** prove that the application's Maven-provided Jakarta artefacts were packaged in the WAR.

For lodash-es:

```
Maven plugin ClassRealm    → YES
source map                 → YES, code contributed
WAR package identity       → NO
Syft final image           → NO
Docker Scout final image   → NO
```

Once esbuild bundled the code, neither final-image scanner reconstructed the npm identity.

Observation	S01	S02
Syft final-image packages	179	589
Docker Scout packages	237	655
Scout provenance attestation	yes	yes
Scout base image	`eclipse-temurin:21-jre-jammy`	`payara/server-web:7.2026.7`
Ordinary app dependency recovered	yes	yes
Shaded Java dependency recovered	yes	n/a
Bundled npm identity recovered	no	no
Runtime/base-image universe exposed	yes	yes
Scout recommendations	Java 25/26 alternatives	none

The absolute package counts differ between Syft and Scout even when both observe the same image.

Therefore:

```
same artefact boundary
    !=
same software inventory
```

Scanner implementation, cataloguers and package-identification rules still matter.

---

1. A CONTAINER IMAGE IS A DIFFERENT EVIDENCE BOUNDARY FROM THE APPLICATION PACKAGE

The final container includes:

```
application software
runtime/server software
JDK/JRE software
OS packages
```

An application POM, npm lockfile, WAR or executable JAR does not describe that complete deployed universe.

2. MOVING LATER IN THE SUPPLY CHAIN CAN ADD SOFTWARE

S02 shows this directly:

```
Jakarta provided dependency
    present in Maven model
    absent from WAR
    present again in final deployment through Payara
```

The same package family can disappear at one boundary and reappear at another for a different reason.

3. MOVING LATER DOES NOT NECESSARILY RECOVER LOST IDENTITY

Both scenarios contain npm code that loses package identity during bundling:

```
S01  lodash
S02  lodash-es
```

Neither Syft nor Docker Scout reconstructed those npm package identities from the final container.

4. DIFFERENT SCANNERS CAN DISAGREE AT THE SAME BOUNDARY

Observed package counts:

```
S01
    Syft          179
    Docker Scout 237

S02
    Syft          589
    Docker Scout 655
```

The final image is the same.

The inventory is not.

5. DOCKER SCOUT ADDS EVIDENCE BEYOND PACKAGE INVENTORY

Observed Scout evidence includes:

```
base-image identification
vulnerability intelligence
policy evaluation
health score
base-image recommendations
build provenance attestation
repository and source commit
```

These are useful deployed-image facts, but they do not reconstruct every earlier build transformation.

6. PROVENANCE AND PACKAGE DISCOVERY ARE SEPARATE EVIDENCE CHANNELS

Scout obtained provenance attestations linking both images to the repository and commit.

That provenance existed separately from ordinary image labels and separately from Scout's package catalogue.

A provenance claim about where an image was built does not itself imply complete knowledge of every dependency or transformation used during the build.

Docker Scout is valuable precisely because it observes a boundary the source dependency tools do not:

```
the final deployed container
```

That gives it visibility into the runtime, server, JDK and OS software universe and allows it to attach vulnerability, policy, recommendation and provenance information.

But T02 also confirms two limits:

```
a later scan cannot be assumed to recover identity destroyed earlier

and

two scanners looking at the same final image need not produce the same invento
```

The practical rule is:

Choose the evidence source for the question you are asking. A container scanner tells you about the deployed container; it does not replace dependency, build-time, transformation or publication evidence collected earlier in the supply chain.

T03 — TRIVY ACROSS S01 BOUNDARIES

OBJECTIVE

Use one vulnerability tool, Trivy, against several evidence boundaries in S01.

The central question is:

> Does the vulnerability answer change when the same software is observed as a dependency model, a transformed Java archive, a browser bundle, or a final container image?

Trivy 0.74.0 was used for the observed run.

For source dependency models and the frontend bundle, the investigation uses:

```
trivy fs
```

For built Java archives it uses:

```
trivy rootfs
```

For the final image it uses:

```
trivy image --image-src docker
```

The distinction matters: passing a JAR directly to `trivy fs` produced:

```
Number of language-specific files num=0
```

and did not invoke the Java archive detector. The corrected archive probes stage each JAR in an isolated directory and scan that directory with `trivy rootfs`.

LOOK

S01 contains these tracer states:

```
jackson-databind 2.19.4
    ordinary Maven dependency
    shipped intact

commons-codec 1.18.0
    service-selected Maven dependency
    shipped intact

commons-codec 1.17.1
    dependency of normalizer
    shaded and relocated
    Maven identity metadata survives in the original shaded JAR

normalizer 1.0.0
    custom application library

lodash 4.17.21
    npm dependency
```

```
    bundled by Vite
    npm package boundary disappears
```

A controlled copy of the shaded normalizer removes only:

```
META-INF/maven/commons-codec/commons-codec/*
```

The relocated codec bytecode is unchanged.

RUN

```
./scripts/baseline-s01.sh
```

OBSERVE

Maven showed:

```
normalizer
    -> commons-codec 1.17.1
```

The service dependency tree showed:

```
service
    -> jackson-databind 2.19.4
    -> normalizer 1.0.0
         -> commons-codec 1.18.0
            (version managed from 1.17.1)
```

npm showed:

```
checkout-trace-frontend@1.0.0
    -> lodash@4.17.21
```

The original shaded normalizer contained relocated codec classes under:

```
com/acme/internal/codec/
```

and retained:

```
META-INF/maven/commons-codec/commons-codec/pom.xml
META-INF/maven/commons-codec/commons-codec/pom.properties
```

The controlled stripped copy still contained the relocated codec bytecode but no codec Maven metadata.

Syft identified the original shaded archive as:

```
commons-codec 1.17.1
normalizer    1.0.0
```

After only codec Maven metadata was removed, Syft no longer identified codec 1.17.1.

The service JAR physically contained:

```
BOOT-INF/lib/jackson-databind-2.19.4.jar
BOOT-INF/lib/normalizer-1.0.0.jar
BOOT-INF/lib/commons-codec-1.18.0.jar
```

and the bundled frontend under:

```
BOOT-INF/classes/static/
```

ESTABLISH

The baseline separates:

```
declared dependency identity
resolved dependency identity
physical shipped bytes
recoverable package identity
```

Those are not the same thing.

RUN

```
./scripts/run-trivy-nonarchives-s01.sh
./scripts/run-trivy-archives-s01.sh
./scripts/compare-s01.sh
```

The full clean harness can also be run with:

```
./scripts/run-trivy-s01.sh
```

BOUNDARY 1 — NORMALIZER POM

Observe

Trivy identified:

```
commons-codec 1.17.1
normalizer    1.0.0
```

No tracer vulnerability finding was reported.

Establish

At the source dependency boundary, Trivy can identify codec 1.17.1 directly from Maven model evidence.

BOUNDARY 2 — SERVICE POM

Observe

Trivy identified:

```
jackson-databind 2.19.4
normalizer       1.0.0
```

It did not surface codec 1.18.0 in the tracer inventory from this single POM scan.

For Jackson 2.19.4 it reported:

```
HIGH    CVE-2026-54512
HIGH    CVE-2026-54513
MEDIUM  CVE-2026-54514
MEDIUM  CVE-2026-54515
MEDIUM  CVE-2026-59888
```

Establish

A source-model vulnerability scan answers a dependency-model question, not necessarily a complete resolved-graph or built-artefact question.

BOUNDARY 3 — FRONTEND PACKAGE-LOCK

Observe

Trivy identified:

```
lodash 4.17.21
```

and reported:

```
HIGH    CVE-2026-4800
MEDIUM  CVE-2025-13465
MEDIUM  CVE-2026-2950
```

Establish

At the npm package-model boundary, lodash has explicit package identity and Trivy can attach vulnerability intelligence to it.

BOUNDARY 4 — SHADED NORMALIZER JAR

Observe

Using `trivy rootfs` against an isolated directory containing the shaded JAR, Trivy identified:

```
commons-codec 1.17.1
normalizer    1.0.0
```

No tracer vulnerability finding was reported.

Establish

Trivy can recover the shaded codec identity from the built artefact while the embedded Maven metadata survives.

BOUNDARY 5 — METADATA-STRIPPED SHADED NORMALIZER JAR

Observe

The archive still contains the same relocated codec bytecode.

Trivy identified only:

```
normalizer 1.0.0
```

It no longer identified:

```
commons-codec 1.17.1
```

Establish

This is the key controlled experiment:

```
same relocated codec bytecode
        +
Maven metadata present
        ↓
commons-codec 1.17.1 identified

same relocated codec bytecode
        +
Maven metadata removed
        ↓
commons-codec identity disappears
```

The software bytes did not disappear.

The package identity did.

A CVE scanner cannot attach package-version vulnerability intelligence to an identity it has not established.

BOUNDARY 6 — SPRING BOOT SERVICE JAR

Observe

Trivy identified:

```
jackson-databind 2.19.4
commons-codec    1.17.1
commons-codec    1.18.0
normalizer       1.0.0
```

For Jackson 2.19.4 it reported the same five CVEs:

```
2 HIGH
3 MEDIUM
```

Establish

The built service JAR contains a software reality that differs from the service Maven model:

```
service Maven model
    -> codec 1.18.0

built service JAR
    -> codec 1.18.0
    -> codec 1.17.1 embedded inside shaded normalizer
```

The shaded codec becomes visible because its identifying Maven metadata survives inside the nested normalizer archive.

BOUNDARY 7 — FRONTEND/DIST

Observe

Trivy reported:

```
Number of language-specific files num=0
```

No tracer identities were recovered.

No tracer vulnerability findings were reported.

Establish

The Vite output ships browser JavaScript, but the npm package boundary has disappeared.

For lodash:

```
package-lock.json
    -> lodash 4.17.21 identified
    -> 3 CVEs identified

frontend/dist
    -> no lodash package identity
    -> no lodash CVE findings
```

This is not evidence that lodash code is absent.

It is evidence that Trivy cannot reconstruct the npm package identity from the bundled output.

BOUNDARY 8 — FINAL CONTAINER IMAGE

Observe

Trivy detected:

```
OS: Ubuntu 22.04
OS packages analysed: 143
Java archive files analysed: 1
```

Tracer identity:

```
jackson-databind 2.19.4
commons-codec    1.17.1
commons-codec    1.18.0
normalizer       1.0.0
```

Lodash was not recovered.

For Jackson 2.19.4 Trivy again reported:

```
HIGH    CVE-2026-54512
HIGH    CVE-2026-54513
MEDIUM  CVE-2026-54514
MEDIUM  CVE-2026-54515
MEDIUM  CVE-2026-59888
```

Establish

The final container exposes a broader deployed software boundary because Trivy also analyses operating-system packages.

But moving later in the supply chain does not restore npm identity that Vite already destroyed.

Boundary	codec 1.17.1	codec 1.18.0	Jackson 2.19.4	lodash 4.17.21
normalizer POM	yes	no	no	no
service POM	no	no	yes	no
package-lock	no	no	no	yes
shaded normalizer JAR	yes	no	no	no
stripped normalizer JAR	no	no	no	no
service JAR	yes	yes	yes	no
frontend/dist	no	no	no	no
final container	yes	yes	yes	no

For the tracer CVEs observed:

```
Jackson 2.19.4
    service POM      -> 5 CVEs
    service JAR      -> 5 CVEs
    final container  -> 5 CVEs

lodash 4.17.21
    package-lock     -> 3 CVEs
    frontend/dist    -> none
    final container  -> none
```

---

1. VULNERABILITY VISIBILITY DEPENDS ON PACKAGE IDENTITY

The controlled codec experiment demonstrates:

```
code present
    !=
package identified
```

```
    !=
CVE can be attached
```

Removing only package metadata changes the scanner's software inventory even though the executable bytecode remains.

2. A SOURCE DEPENDENCY MODEL AND A BUILT ARTEFACT ANSWER DIFFERENT QUESTIONS

The normalizer POM says:

```
commons-codec 1.17.1
```

The service Maven model says:

```
commons-codec 1.18.0
```

The built service JAR exposes:

```
commons-codec 1.17.1
commons-codec 1.18.0
```

All three are valid observations at different supply-chain boundaries.

3. TRANSFORMATION CAN REMOVE VULNERABILITY VISIBILITY WITHOUT REMOVING CODE

Lodash demonstrates this independently of Java shading:

```
npm lockfile
    -> identity present
    -> CVEs present

Vite bundle
    -> code still shipped
    -> npm identity absent
    -> CVEs absent
```

The absence of a CVE finding at the later boundary does not prove absence of the vulnerable code.

4. MOVING THE SCANNER LATER GIVES A BROADER DEPLOYED VIEW, BUT NOT COMPLETE HISTORY

The container scan adds Ubuntu package analysis and sees the final application archive.

It still does not reconstruct lodash identity after bundling.

The final image therefore answers:

```
what software can Trivy identify in this deployment?
```

not:

```
what software participated in every earlier build transformation?
```

5. SCANNER MODE IS PART OF THE EVIDENCE MODEL

For Trivy 0.74.0:

```
trivy fs
    -> source/dependency model and filesystem-oriented scanning

trivy rootfs
    -> post-build filesystem/archive analysis

trivy image
    -> final container analysis
```

Using the wrong scan mode can produce a false negative experiment before vulnerability matching even begins.

T03 demonstrates that a CVE scanner is downstream of software identification.

The practical chain is:

```
bytes exist
    ↓
scanner recognises package identity
    ↓
scanner determines package version
    ↓
vulnerability database matches affected range
    ↓
CVE appears
```

Break the chain at package identity and the CVE can disappear from the result even though the underlying code remains.

The practical rule is:

> A clean vulnerability scan proves only that the scanner found no matching vulnerabilities for the software identities it was able to establish at that evidence boundary. It does not prove that all shipped software was identified.

T04 — GRYPE / S02

OBJECTIVE

Use Grype against S02 to separate:

```
software inventory construction
```

from:

```
vulnerability matching
```

The central question is:

> Does Grype produce a different vulnerability answer when it discovers the Payara image itself versus when it consumes an SBOM generated from that same image?

The observed run used:

```
Grype 0.115.0
Syft 1.46.0
Grype DB schema 6.1.9
Grype DB built 2026-08-22T06:14:16Z
DB status: valid
```

The same final image and the same Grype vulnerability database were used throughout.

LOOK

S02 has three deliberately different kinds of software evidence.

Application dependencies

The Maven application model contains:

```
jakarta.platform:jakarta.jakartaee-web-api:11.0.0   provided
org.apache.commons:commons-lang3:3.18.0             compile
```

Maven plugin dependencies

The `esbuild-maven-plugin` runs during `generate-resources`.

Its Maven plugin realm contains:

```
io.mvnpm:esbuild-maven-plugin:2.0.0
org.mvnpm:lodash-es:4.17.21
```

The plugin realm also contains its own larger transitive toolchain, including:

```
commons-codec 1.19.0
commons-lang3 3.18.0
jackson-databind 2.20.1
jackson-core 2.20.1
```

WAR contents

The built WAR contains:

```
WEB-INF/lib/commons-lang3-3.18.0.jar
assets/app.js
assets/app.js.map
```

It does not physically package the provided Jakarta EE API dependency.

It does not contain a `lodash-es` package boundary.

RUN

```
./scripts/baseline-s02.sh
```

OBSERVE

The final image is:

```
payara-mvnpm-trace-lab:local
```

Observed image digest:

```
sha256:9deab630f88089a2254668ed55569a73f0317a0d1d226832ed1134436a48aeaa
```

Syft catalogued:

```
589 packages
825 executables
5,424 locations
```

The Syft image inventory identifies:

```
commons-lang3 3.18.0
payara-mvnpm-trace-lab 1.0.0
many Jakarta API/runtime components supplied by Payara
```

It does not identify:

```
lodash-es 4.17.21
```

The generated CycloneDX document contained:

```
6014 components
```

This number is not treated as equivalent to the Syft package count. The CycloneDX document represents more than the 589-package catalogue.

ESTABLISH

S02 separates four boundaries:

```
application dependency model
Maven plugin realm
WAR contents
final deployed container
```

They contain different software universes.

RUN

```
./scripts/run-grype-s02.sh
```

Grype was run three ways:

```
1. docker:payara-mvnpm-trace-lab:local

2. sbom:results/s02/baseline/image.syft.json

3. sbom:results/s02/baseline/image.cdx.json
```

Two direct PURL controls were also used:

```
pkg:maven/org.apache.commons/commons-lang3@3.18.0
pkg:maven/org.mvnpm/lodash-es@4.17.21
```

OBSERVE

The first attempted run failed because the local Grype database was too old:

```
Status: invalid
the vulnerability database was built 2 weeks ago
(max allowed age is 5 days)
```

After refreshing the database, the successful run reported:

```
Schema: v6.1.9
Built: 2026-08-22T06:14:16Z
Status: valid
```

ESTABLISH

Vulnerability database state is part of the evidence.

A stale or invalid database can prevent the vulnerability experiment from running at all.

OBSERVE

Grype reported:

```
169 unique vulnerability matches
```

The findings include both operating-system and Java-runtime packages.

Examples from the final image include:

```
nimbus-jose-jwt 10.0.1
    GHSA-xwmg-2g98-w7v9
    Medium
    fixed in 10.0.2

jline-remote-telnet 3.30.15
    GHSA-2r2c-cx56-8933
    High
    fixed in 4.2.1

jline-remote-telnet 3.30.15
    GHSA-47qp-hqvx-6r3f
    High
    fixed in 4.2.1

jackson-core 2.15.2
    GHSA-r7wm-3cxj-wff9
    High
    fixed in 2.18.8

jackson-core 2.15.2
    GHSA-72hv-8253-57qq
    Medium
    fixed in 2.18.6
```

There were also many Ubuntu package findings.

No tracer-related vulnerability match was reported for:

```
commons-lang3
lodash-es
Payara-named tracer pattern
Jakarta-named tracer pattern
```

ESTABLISH

The final-image vulnerability answer is dominated by the deployed runtime/base-image software universe, not merely the application WAR.

The harness did not expose a direct Grype package-catalogue count, so this investigation does not claim one.

OBSERVE

Grype emitted:

```
document has schema version 16.1.10,
but parser has older schema version 16.1.5
```

Despite that compatibility warning, Grype again produced:

```
169 unique vulnerability matches
```

ESTABLISH

For this image and these tool versions, the Syft schema-version warning did not change the resulting vulnerability match set.

OBSERVE

Grype consumed the CycloneDX SBOM generated from the same image and again produced:

```
169 unique vulnerability matches
```

ESTABLISH

The fact that the CycloneDX document contains 6014 components rather than the Syft JSON's 589 package artifacts did not produce a different vulnerability answer in this run.

Those raw document counts therefore should not be interpreted as equivalent package counts.

RUN

```
./scripts/compare-s02.sh
```

OBSERVE

The final comparison showed:

```
direct image     169 unique vulnerability matches
Syft JSON        169 unique vulnerability matches
CycloneDX        169 unique vulnerability matches
```

and:

```
direct image vs Syft JSON
    no differences

direct image vs CycloneDX
    no differences

Syft JSON vs CycloneDX
    no differences
```

ESTABLISH

For this S02 image:

```
Grype direct discovery
        =
Grype over Syft JSON
        =
Grype over CycloneDX
```

with respect to the vulnerability match set.

The inventory representation did not change Grype's vulnerability answer.

OBSERVE

The final image inventory contains:

```
commons-lang3 3.18.0
```

The direct PURL control produced:

```
no vulnerability matches
```

The image/SBOM scans likewise produced no tracer-related vulnerability match for commons-lang3.

ESTABLISH

This is not an inventory failure.

Grype identifies the package in the final image inventory, but the observed Grype vulnerability database produced no match for that package/version.

OBSERVE

Maven plugin evidence proves:

```
org.mvnpm:lodash-es:4.17.21
```

was included in the `esbuild-maven-plugin` ClassRealm during the build.

The final WAR contains the generated browser bundle, but no `lodash-es` package boundary.

The final image Syft inventory does not identify lodash-es.

The direct PURL control also produced:

```
no vulnerability matches
```

ESTABLISH

For this run, lodash-es cannot demonstrate:

```
Grype knows the vulnerability
but package discovery loses the package
```

because the direct PURL control produced no vulnerability match.

What it does demonstrate is:

```
build participation
    !=
final-image package identity
```

Grype cannot vulnerability-match a final-image package identity that is not present in the supplied inventory.

OBSERVE

The application Maven model declares:

```
jakarta.jakartaee-web-api 11.0.0
    scope: provided
```

The WAR does not package that API dependency.

The final image nevertheless contains many Jakarta components, including:

```
jakarta.servlet-api 6.1.0
jakarta.ws.rs-api 4.0.0
jakarta.persistence-api 3.2.0
jakarta.validation-api 3.1.1
...
```

Those components are supplied by the Payara runtime image.

ESTABLISH

The container vulnerability scan answers:

```
what software is deployed together?
```

It does not tell us that all software found in the image came from the application dependency graph or WAR.

---

1. GRYPE CAN CONSUME DIFFERENT INVENTORY REPRESENTATIONS WITHOUT CHANGING THE ANSWER

In this experiment:

```
direct image
Syft JSON
CycloneDX
```

all produced exactly:

```
169 unique vulnerability matches
```

with no match-set differences.

This means the SBOM representation was not the source of vulnerability-answer variation here.

2. INVENTORY STILL DETERMINES WHAT CAN BE VULNERABILITY-MATCHED

The equality of the three Grype results does not mean the inventory is complete.

All three views ultimately describe the same final-image software boundary.

They all omit `lodash-es` identity after its build-time contribution has been bundled away.

3. A CONTAINER VULNERABILITY SCAN INCLUDES SOFTWARE THE APPLICATION DID NOT PACKAGE

The application WAR contains commons-lang3 and generated browser assets.

The final container also contains:

```
Ubuntu OS packages
Payara/GlassFish runtime libraries
Jakarta APIs
server-side Jackson
JLine
Nimbus JOSE JWT
...
```

Those packages materially change the deployed vulnerability surface.

4. BUILD-TIME SOFTWARE CAN DISAPPEAR BEFORE FINAL-IMAGE VULNERABILITY SCANNING

`lodash-es 4.17.21` is proven in the Maven plugin realm.

Its package identity does not survive into the WAR or final image inventory.

Therefore:

```
software participated in producing the artefact
```

does not imply:

```
final-image vulnerability scanner can identify that software
```

5. THE VULNERABILITY DATABASE IS ITSELF PART OF THE RESULT

The initial run could not proceed because the DB was invalid due to age.

After updating it, the scan succeeded.

A CVE scan therefore depends on at least:

```
inventory evidence
package identity
version identity
vulnerability database content
database freshness/validity
matching rules
```

T04 demonstrates that changing the *representation* of a good final-image inventory does not necessarily change the vulnerability answer.

For this image:

```
direct image discovery
        =
Syft JSON
```

```
        =
CycloneDX
```

at the Grype vulnerability-match layer.

But all three still inherit the same supply-chain limitation:

> They can only vulnerability-match software identities that survived into, or can be reconstructed from, the final-image evidence.

The practical rule is:

> An SBOM-driven CVE scan can be perfectly consistent with a direct image scan and still miss software that disappeared as an identifiable package at an earlier build boundary.

T05 — PIP-AUDIT / S03

OBJECTIVE

Use `pip-audit` against S03 to separate four different facts:

```
dependency identity
vulnerability matching
PEP 517 build execution
generated installed content
```

The central question is:

> Can a Python vulnerability audit reconstruct the fact that a transitive source distribution executed a PEP 517 backend and generated runtime files?

T05 also asks a more surprising question:

> Can running the vulnerability audit itself execute packaging code from the dependency being audited?

The observed answer to the second question is **yes.**

```
pip-audit 2.10.1
Python 3.14
pip 26.1.2
```

LOOK

The direct requirement is:

```
reportkit==1.0.0
```

The `reportkit` wheel metadata contains:

```
Requires-Dist: tracehook-demo==1.0.0
```

`tracehook-demo` is supplied as a source distribution.

Its source distribution contains only:

```
tracehook_demo-1.0.0/pyproject.toml
tracehook_demo-1.0.0/tracehook_backend.py
```

Its PEP 517 configuration contains:

```
[build-system]
requires = []
build-backend = "tracehook_backend"
backend-path = ["."]
```

The source distribution does **not** contain:

```
tracehook_demo/__init__.py
tracehook_demo/build-hook.json
```

RUN

```
./scripts/baseline-s03.sh
```

OBSERVE

Ordinary pip installation showed:

```
Processing reportkit-1.0.0-py3-none-any.whl

Processing tracehook_demo-1.0.0.tar.gz
  Getting requirements to build wheel
  Preparing metadata (pyproject.toml)

Building wheels for collected packages: tracehook-demo
  Building wheel for tracehook-demo (pyproject.toml)
  Created wheel for tracehook-demo

Successfully installed:
  reportkit-1.0.0
  tracehook-demo-1.0.0
```

The installed environment contained:

```
pip             26.1.2
reportkit       1.0.0
tracehook-demo  1.0.0
```

The installed package contained files absent from the source distribution:

```
tracehook_demo/__init__.py
tracehook_demo/build-hook.json
```

The generated marker contained:

```
{
  "event": "pep517-build-backend-executed",
  "generatedBy": "tracehook_backend.build_wheel",
  "package": "tracehook-demo",
  "version": "1.0.0"
}
```

ESTABLISH

The package installation crosses a transformation boundary:

```
sdist
    ↓ PEP 517 backend execution
wheel
```

```
    ↓ install
runtime files
```

Therefore:

```
source distribution contents
    !=
installed runtime contents
```

RUN

```
./scripts/run-pip-audit-s03.sh
```

The normal requirements audit uses dependency resolution.

OBSERVE

`pip-audit` reported:

```
Dry run: would have audited 2 packages
```

The resulting audit set contained:

```
reportkit
tracehook-demo
```

Both were skipped with:

```
Dependency not found on PyPI and could not be audited
```

ESTABLISH

`pip-audit` successfully recovered the transitive dependency identity:

```
reportkit
    → tracehook-demo
```

But package identity did not imply vulnerability coverage.

For these private scenario packages:

```
package identified
    !=
package vulnerability-auditable through PyPI
```

LOOK

The harness creates a temporary, controlled copy of the same `tracehook-demo` source distribution.

The dependency metadata is unchanged.

The backend receives one benign observable side effect:

```
when tracehook_backend is imported
    write a marker file
```

RUN

```
./scripts/run-pep517-exec-probe.sh
```

OBSERVE

The audit reported:

```
INFO:pip_audit._audit:Dry run: would have audited 2 packages
No known vulnerabilities found
```

The marker was created:

```
PEP 517 execution marker:

tracehook_backend imported during pip-audit dependency resolution
```

ESTABLISH

This is direct evidence that:

```
pip-audit -r requirements.txt
    ↓
pip-assisted dependency collection
    ↓
PEP 517 hook processing
    ↓
import tracehook_backend
    ↓
backend code executes
```

The important point is not that `pip-audit` maliciously executes code.

The important point is that a vulnerability audit of a requirements file can cross a Python packaging execution boundary because dependency collection uses pip and PEP 517 machinery.

So:

> Running the security audit can itself execute code from the dependency being audited.

This is not visible in the final vulnerability report.

OBSERVE

With:

```
pip-audit --no-deps ...
```

the observed audit set still contained:

```
reportkit
tracehook-demo
```

The tool emitted:

```
--no-deps is supported, but users are encouraged to fully hash their pinned de
```

ESTABLISH

For the observed `pip-audit 2.10.1` run, `--no-deps` alone did not reduce the resulting audit set to only the directly pinned requirement.

That is recorded as observed behaviour rather than inferred behaviour.

OBSERVE

With:

```
pip-audit --no-deps --disable-pip ...
```

the audit set contained only:

```
reportkit
```

`tracehook-demo` was absent.

ESTABLISH

The difference is:

```
--no-deps
    reportkit
    tracehook-demo

--no-deps --disable-pip
    reportkit
```

This isolates pip-assisted dependency collection from static handling of the explicitly pinned requirements file.

RUN

The harness audits the already-installed `site-packages` directory.

OBSERVE

The audit saw:

```
pip 26.1.2
reportkit 1.0.0
tracehook-demo 1.0.0
```

It found one known vulnerability:

```
pip 26.1.2

PYSEC-2026-3721
CVE-2026-13346

fixed in:
26.2
```

The scenario packages were again skipped because they were not found in PyPI's vulnerability service.

ESTABLISH

Auditing the installed environment changes the software universe.

The vulnerability scan now includes packaging/runtime tooling present in that environment:

```
pip
```

That produced a real vulnerability finding that was unrelated to the application dependency graph itself.

OBSERVE

The observed pip-audit CycloneDX output surfaced:

```
pip 26.1.2
```

The comparison helper did not surface the skipped private packages as CycloneDX components:

```
reportkit
tracehook-demo
```

ESTABLISH

The JSON vulnerability result and the CycloneDX output preserve different evidence.

The JSON result explicitly records private packages as:

```
identified but skipped
```

The observed CycloneDX component view did not retain those identities.

Therefore:

```
same audit operation
    !=
same evidence preserved in every output format
```

---

1. DEPENDENCY RESOLUTION CAN RECOVER A TRANSITIVE PACKAGE

The direct input declared only:

```
reportkit==1.0.0
```

Normal pip-audit resolution recovered:

```
tracehook-demo==1.0.0
```

So transitive dependency identity can be reconstructed from package metadata.

---

2. IDENTIFIED PACKAGES CAN STILL BE UNAUDITABLE

Both scenario packages were identified but skipped:

```
Dependency not found on PyPI and could not be audited
```

This is materially different from:

```
package not identified
```

A clean result for a skipped/private package is not proof that the package has no vulnerabilities.

---

3. A REQUIREMENTS VULNERABILITY AUDIT CAN EXECUTE PACKAGING CODE

The controlled marker proves:

```
pip-audit
    → pip dependency collection
    → PEP 517 backend import
    → backend code execution
```

This is the strongest T05 result.

The vulnerability audit itself participates in the Python software supply chain.

---

4. VULNERABILITY OUTPUT DOES NOT PRESERVE PEP 517 EXECUTION HISTORY

Although the backend executed during dependency resolution, the vulnerability result does not say:

```
tracehook_backend was imported
PEP 517 hooks ran
runtime files were generated
```

That evidence exists only because the build/install boundary was separately instrumented.

---

5. INSTALLED PACKAGE IDENTITY DOES NOT EXPLAIN HOW INSTALLED FILES WERE CREATED

The installed environment knows:

```
tracehook-demo 1.0.0
```

But that identity alone does not tell us that:

```
__init__.py
build-hook.json
```

were generated by the backend and were absent from the source distribution.

So:

```
installed package identity
    !=
installation provenance
```

---

6. --DISABLE-PIP MATERIALLY CHANGES THE OBSERVED AUDIT BOUNDARY

Observed:

```
--no-deps
    reportkit
    tracehook-demo

--no-deps --disable-pip
    reportkit
```

That distinction matters because using pip for collection can cross packaging execution boundaries.

---

7. AUDITING THE ENVIRONMENT CAN REVEAL VULNERABILITIES IN THE AUDIT/INSTALL TOOLING ITSELF

The installed-environment scan found:

```
pip 26.1.2
CVE-2026-13346
```

So the vulnerability surface of the environment includes more than the application's declared packages.

T05 demonstrates two different supply-chain blind spots.

First:

```
package identity
    !=
vulnerability coverage
```

`reportkit` and `tracehook-demo` were identified but not auditable through the selected vulnerability service.

Second:

```
package identity
    !=
build/install history
```

`pip-audit` could identify the installed/transitive packages, but its final report did not preserve the fact that PEP 517 code executed or that runtime files were generated during wheel construction.

Most importantly, the audit operation itself can participate in that execution chain:

```
audit requirements
    → resolve dependencies
    → invoke packaging machinery
    → execute PEP 517 backend code
```

The practical rule is:

> Treat Python dependency auditing as an active supply-chain operation when dependency resolution is enabled, not as a purely static inspection step.

T06 — OWASP DEPENDENCY-CHECK / S04

OBJECTIVE

Use OWASP Dependency-Check against S04 to distinguish:

```
application dependency inventory
Maven plugin dependency inventory
plugin execution
final application bytes
vulnerability matching
```

The central question is:

> If a Maven plugin and its transitive dependency generate runtime capability that survives in the final application JAR, at which boundary can Dependency-Check still identify the build-time packages?

```
OWASP Dependency-Check Maven Plugin 13.0.0
```

The successful walkthrough used:

```
NVD API key: supplied via NVD_API_KEY environment variable
```

Dependency-Check data directory:

```
~/.cache/kcdc-dependency-check/13.0.0
```

WHY THIS MATTERS

OWASP Dependency-Check maintains a local vulnerability database populated from NVD data.

An NVD API key is not conceptually required for Dependency-Check, but version 13.0.0 has a known no-key regression in its NVD update path. The T06 walkthrough therefore used a valid NVD API key with 13.0.0.

The NVD documents higher request limits when a key is supplied.

For a workshop, the best option is normally to reuse a pre-populated Dependency-Check data directory so every attendee does not have to populate the complete NVD database.

GETTING AN NVD API KEY

Go to:

```
https://nvd.nist.gov/developers/request-an-api-key
```

The request form asks for:

```
organisation name
email address
organisation type
```

Then:

```
1. Read and accept the NVD Terms of Use.
2. Submit the request.
3. Check the supplied email address for the NVD activation email.
4. Open the single-use activation link.
5. Activate and view the API key.
6. Store the key somewhere secure.
```

The activation link expires after seven days.

The page used to display the key is single-use, so copy it to a secure secret store when it is displayed.

Requesting and activating another key with the same email address invalidates the previous key.

Do not commit an NVD API key to this repository.

USING THE KEY WITH T06

Export it into the environment:

```
export NVD_API_KEY='your-key-here'
./scripts/run-dependency-check-s04.sh
```

Check that it is actually exported:

```
printenv NVD_API_KEY >/dev/null && echo "NVD_API_KEY is exported"
```

The harness should print:

```
NVD API key: supplied via NVD_API_KEY environment variable
```

RUN

```
./scripts/baseline-s04.sh
```

OBSERVE

The application dependency tree contained only the application itself:

```
dev.noregressions.trace:maven-plugin-hidden-content:jar:1.0.0
```

No normal application dependency on either controlled build-time tracer was present.

ESTABLISH

The ordinary Maven dependency universe is:

```
maven-plugin-hidden-content 1.0.0
```

with no application dependency on:

```
trace-injector-maven-plugin
trace-route-payload
```

OBSERVE

Maven plugin resolution showed:

```
dev.noregressions.trace:trace-injector-maven-plugin:maven-plugin:1.0.0:runtime
    dev.noregressions.trace:trace-injector-maven-plugin:jar:1.0.0
    dev.noregressions.trace:trace-route-payload:jar:1.0.0
```

ESTABLISH

Maven has a separate build-tooling dependency domain:

```
trace-injector-maven-plugin 1.0.0
    ↓
trace-route-payload 1.0.0
```

Neither package is an application dependency.

OBSERVE

Maven debug output showed:

```
Created new class realm plugin>dev.noregressions.trace:trace-injector-maven-pl

Included:
dev.noregressions.trace:trace-injector-maven-plugin:jar:1.0.0
dev.noregressions.trace:trace-route-payload:jar:1.0.0
```

Maven then loaded:

```
trace-injector-maven-plugin:1.0.0:inject-route
```

from that ClassRealm.

ESTABLISH

The payload was not merely resolvable.

It was present in the actual Maven ClassRealm used to execute the build plugin.

OBSERVE

Plugin execution generated:

```
target/generated-sources/trace-injector/.../GeneratedTraceRoute.java

target/generated-resources/trace-injector/
```

```
    META-INF/services/dev.noregressions.trace.s04.TraceRoute
    META-INF/trace-lab/plugin-injection.properties
```

The final JAR contained:

```
META-INF/trace-lab/plugin-injection.properties
META-INF/services/dev.noregressions.trace.s04.TraceRoute
dev/noregressions/trace/s04/generated/GeneratedTraceRoute.class
```

Disassembly of the final class contained:

```
/hidden/build-info
trace-route-payload
trace-injector-maven-plugin
```

ESTABLISH

The build-time packages changed the final runtime capability.

The transformation was:

```
plugin + payload
    ↓ Maven ClassRealm
plugin execution
    ↓
generated Java + ServiceLoader metadata
    ↓ compile/package
final application JAR
```

The behaviour survives into the final JAR.

RUN

```
./scripts/run-dependency-check-s04.sh
```

OBSERVE

The default scan produced:

```
Dependencies: 0
Vulnerability records: 0
S04 tracers:
(none)
```

ESTABLISH

The default Maven Dependency-Check view followed the ordinary application dependency model.

It did not include:

```
trace-injector-maven-plugin
trace-route-payload
```

So:

```
default application scan
    !=
complete Maven build-tooling inventory
```

OBSERVE

With Maven plugin scanning enabled, Dependency-Check reported:

```
Dependencies: 167
Vulnerability records: 78
```

It identified both controlled S04 tracers:

```
trace-injector-maven-plugin-1.0.0.jar
pkg:maven/dev.noregressions.trace/trace-injector-maven-plugin@1.0.0

trace-route-payload-1.0.0.jar
pkg:maven/dev.noregressions.trace/trace-route-payload@1.0.0
```

Neither tracer had a vulnerability match.

The plugin-aware scan also exposed vulnerabilities in build-time tooling, including dependencies such as:

```
commons-beanutils 1.7.0
commons-compress 1.20
commons-io 2.6
commons-io 2.11.0
guava 16.0.1
maven-core 3.2.5
jetty 9.4.46.v20220331
plexus-archiver 4.2.7
velocity 1.7
```

ESTABLISH

Changing only the evidence admitted to Dependency-Check changed the software universe from:

```
0 dependencies
0 vulnerability records
```

to:

```
167 dependencies
78 vulnerability records
```

This is the strongest T06 result.

The difference is not a different vulnerability database.

It is a different dependency boundary.

OBSERVE

The final JAR scan produced:

```
Dependencies: 1
Vulnerability records: 0
```

The only S04 identity recovered was:

```
maven-plugin-hidden-content-1.0.0.jar
pkg:maven/dev.noregressions.trace/maven-plugin-hidden-content@1.0.0
```

The final JAR scan did not identify:

```
trace-injector-maven-plugin
trace-route-payload
```

ESTABLISH

The final application JAR retains the generated behaviour:

```
GeneratedTraceRoute.class
ServiceLoader metadata
/hidden/build-info
trace-route-payload
trace-injector-maven-plugin
```

but it no longer retains those original build-time components as package identities.

So:

```
runtime behaviour present
    !=
build-time package identity recoverable
```

OBSERVE

Scanning the original build-time JARs directly produced:

```
Dependencies: 2
Vulnerability records: 0
```

Dependency-Check identified both:

```
trace-injector-maven-plugin-1.0.0.jar
pkg:maven/dev.noregressions.trace/trace-injector-maven-plugin@1.0.0

trace-route-payload-1.0.0.jar
pkg:maven/dev.noregressions.trace/trace-route-payload@1.0.0
```

ESTABLISH

This rules out:

```
Dependency-Check cannot recognise the controlled tracer JARs
```

The scanner can identify both original packages when the package boundaries are still available.

The loss occurs after the plugin transformation:

```
original plugin/payload JARs
    → identities recoverable

final application JAR
    → identities not recoverable
```

Observed:

```
default Maven
    no tracers

plugin-aware Maven
    trace-injector-maven-plugin
    trace-route-payload

final JAR
    maven-plugin-hidden-content only

direct plugin/payload
    trace-injector-maven-plugin
    trace-route-payload
```

The transformation boundary is therefore visible directly in the Dependency-Check results.

The first full 13.0.0 run populated:

```
381,857 NVD records
```

and took substantially longer than subsequent runs.

Once populated, later scans reused the local database and completed quickly.

The first run also encountered unrelated RetireJS and .NET Assembly analyzer noise. The final harness disables those analyzers because they are outside the Java-only experiment.

A separate 13.0.0 no-key run failed before analysis with:

```
NvdApiException:
Invalid API Key, length of 0 too short to provided a masked partial key
```

That failure was not an S04 dependency-analysis result.

---

1. MAVEN HAS MORE THAN ONE DEPENDENCY UNIVERSE

The ordinary application dependency graph can be empty while Maven still loads a substantial amount of executable third-party software to build the application.

For S04:

```
application dependencies
    0

plugin-aware Dependency-Check inventory
    167
```

---

2. BUILD TOOLING HAS ITS OWN VULNERABILITY SURFACE

Enabling plugin scanning exposed:

```
78 vulnerability records
```

that were completely absent from the default application scan.

Therefore:

```
application vulnerability scan
    !=
build-pipeline vulnerability scan
```

---

3. MAVEN PLUGIN DEPENDENCIES CAN DIRECTLY CHANGE SHIPPED RUNTIME BEHAVIOUR

trace-route-payload was:

```
not an application dependency
present in the Maven plugin ClassRealm
used during plugin execution
responsible for generated runtime content
```

The final application exposes:

```
/hidden/build-info
```

because of that build-time dependency.

---

4. A FINAL ARTEFACT SCAN CANNOT RECONSTRUCT EVERY BUILD-TIME DEPENDENCY

The final JAR still contains the generated behaviour, but Dependency-Check identified only:

```
maven-plugin-hidden-content 1.0.0
```

It did not reconstruct:

```
trace-injector-maven-plugin
trace-route-payload
```

So:

```
shipped behaviour
    !=
recoverable build provenance
```

---

5. DIRECT CONTROLS PROVE THIS IS IDENTITY LOSS, NOT SCANNER INCAPABILITY

When shown the original plugin and payload JARs directly, Dependency-Check identified both.

Therefore the missing identities in the final application JAR are caused by the transformation boundary, not by an inability to recognise the packages.

---

6. VULNERABILITY VISIBILITY DEPENDS ON WHAT ENTERS THE INVENTORY

The same Dependency-Check engine and vulnerability database produced:

```
default Maven
    0 vulnerabilities

plugin-aware Maven
    78 vulnerability records
```

The decisive variable was not the CVE data.

It was what software identities the scan admitted.

T06 demonstrates:

```
application dependency graph
    !=
build-tool dependency graph
    !=
plugin execution realm
    !=
final artefact package identity
```

and:

```
software executed during build
    can alter shipped runtime behaviour
    without remaining identifiable
    in the final application artefact
```

The practical rule is:

> Scan build tooling at the point where its package identity still exists. A later scan of the shipped application cannot reliably reconstruct the software that transformed it.

T07 — NPM AUDIT / S05

OBJECTIVE

Use `npm audit` against S05 to distinguish:

```
npm dependency metadata
package source contents
npm lifecycle execution
published package contents
generated runtime provenance
vulnerability matching
```

The central question is:

> When `prepack` generates runtime JavaScript that survives publication while the generator and original input disappear, what can `npm audit` still see?

```
Node v26.4.0
npm 12.0.1
registry https://registry.npmjs.org/
```

RUN

```
./scripts/baseline-s05.sh
```

OBSERVE

Before packing, the package source contained:

```
packages/trace-route-package/build-input/route.json
packages/trace-route-package/package.json
packages/trace-route-package/scripts/generate-dist.js
```

Its `package.json` declares:

```
{
  "name": "trace-route-package",
  "version": "1.0.0",
  "main": "dist/index.js",
  "prepack": "node scripts/generate-dist.js"
}
```

ESTABLISH

Before publication, S05 has:

```
package metadata
generator code
```

```
generator input
```

but the runtime `dist/` output has not yet been created.

OBSERVE

`npm pack` printed:

```
npm notice run trace-route-package@1.0.0 prepack
npm notice run node scripts/generate-dist.js
generated dist/index.js for /hidden/prepack-info
generated dist/prepack-evidence.json
```

ESTABLISH

Packaging is an active transformation:

```
package source
    ↓ npm prepack
scripts/generate-dist.js executes
    ↓
dist/index.js
dist/prepack-evidence.json
```

The published runtime bytes are not simply a copy of the original package source.

OBSERVE

The tarball contained exactly:

```
package/dist/index.js
package/package.json
package/dist/prepack-evidence.json
```

It did not contain:

```
scripts/generate-dist.js
build-input/route.json
```

The published `package.json` still retained the lifecycle declaration:

```
"prepack": "node scripts/generate-dist.js"
```

even though the referenced script itself was no longer in the package.

ESTABLISH

Publication preserved:

```
generated runtime bytes
generated evidence file
```

```
package metadata
lifecycle declaration string
```

while discarding:

```
generator implementation
original build input
```

So even before vulnerability scanning:

```
published package
    !=
source package
```

OBSERVE

The installed generated evidence file contained:


```
{
  "event": "npm-prepack-generated",
  "package": "trace-route-package",
  "version": "1.0.0",
  "generatedBy": "npm lifecycle prepack -> scripts/generate-dist.js",
  "message": "This runtime route was generated by an npm prepack lifecycle scr
  "origin": "trace-route-package build input",
  "route": "/hidden/prepack-info"
}
```


ESTABLISH

The final installed package contains explicit evidence that:

```
prepack executed
generator produced runtime content
runtime route is /hidden/prepack-info
```

This gives us a controlled provenance signal to search for in `npm audit`.

RUN

```
./scripts/run-npm-audit-s05.sh
```

OBSERVE

The application audit produced:

```
json=true
vulnerabilityRecords=0
dependencyTotal=1
vulnerabilities:
```

```
(none)
exit=0
```

The installed application dependency tree established by the baseline was:

```
node-prepack-trace-lab@1.0.0
└── trace-route-package@1.0.0
```

ESTABLISH

`npm audit` operated on an npm dependency model containing one dependency.

It reported no vulnerability for that dependency.

The result did not expose how the installed package's runtime bytes had been created.

OBSERVE

Running:

```
npm audit --package-lock-only
```

produced exactly the same audit summary:

```
vulnerabilityRecords=0
dependencyTotal=1
exit=0
```

ESTABLISH

Ignoring the installed `node_modules` tree did not change the result.

For this scenario:

```
application audit
    ==
package-lock-only audit
```

This is consistent with the vulnerability decision being driven by npm dependency metadata rather than inspection of the generated installed files.

OBSERVE

A normal audit in the source package failed:

```
npm error code ENOLOCK
npm error audit This command requires an existing lockfile.
npm error audit Try creating one first with: npm i --package-lock-only
npm error audit Original error: loadVirtual requires existing package-lock.jso
```

The captured result was:

```
dependencyTotal=unknown
vulnerabilityRecords=0
```

```
exit=1
```

ESTABLISH

The source package's physical contents are not sufficient input for a normal `npm audit`.

It contains:

```
package.json
scripts/generate-dist.js
build-input/route.json
```

but without a lockfile npm refuses the ordinary audit.

So:

```
files present on disk
    !=
auditable npm dependency model
```

OBSERVE

Running the source package with:

```
--no-package-lock
```

produced:

```
dependencyTotal=0
vulnerabilityRecords=0
exit=0
```

ESTABLISH

When npm rebuilt the model from the source package metadata, it found no dependencies to audit.

The presence of:

```
scripts/generate-dist.js
build-input/route.json
prepack lifecycle declaration
```

did not create vulnerability inventory entries.

OBSERVE

A normal audit of the unpacked published tarball failed with the same `ENOLOCK` condition:

```
This command requires an existing lockfile.
```

Captured summary:

```
dependencyTotal=unknown
vulnerabilityRecords=0
exit=1
```

ESTABLISH

The fact that the package now physically contains generated runtime JavaScript does not make it independently auditable by `npm audit`.

Again:

```
shipped JavaScript bytes
    !=
npm dependency model
```

OBSERVE

Running the published package with:

```
--no-package-lock
```

produced:

```
dependencyTotal=0
vulnerabilityRecords=0
exit=0
```

ESTABLISH

This is particularly useful because the source and published packages contain materially different bytes:

```
source package
    scripts/generate-dist.js
    build-input/route.json

published package
    dist/index.js
    dist/prepack-evidence.json
```

Yet their `--no-package-lock` audit results were identical:

```
dependencyTotal=0
vulnerabilityRecords=0
```

So `npm audit` did not distinguish those two physical software states.

OBSERVE

Across all S05 audit outputs:

```
scripts/generate-dist.js     not found
npm-prepack-generated        not found
prepack-evidence.json        not found
/hidden/prepack-info         not found
```

ESTABLISH

The source and installed package contain those provenance signals, but the audit output does not expose them.

This is the key T07 distinction:

```
npm lifecycle provenance
    !=
npm vulnerability evidence
```

`npm audit` did not reconstruct:

```
which lifecycle script executed
which generator produced the bytes
which build input was consumed
which runtime route was generated
```

OBSERVE

The isolated public control used:

```
lodash 4.17.21
```

The audit produced:

```
json=true
vulnerabilityRecords=1
dependencyTotal=1
exit=1
```

The vulnerability entry was:

```
lodash
severity: high
direct
range: <=4.17.23
```

and referenced three advisory causes in the returned `via` data.

ESTABLISH

The npm registry audit path was functioning and capable of returning current vulnerability intelligence.

Therefore the clean S05 results were not caused by a broken audit service.

The control establishes:

```
known vulnerable npm dependency identity
    → vulnerability result
```

while S05 establishes:

```
generated runtime provenance
    → no vulnerability inventory entry by itself
```

Observed:

```
application
    dependencyTotal=1
    vulnerabilityRecords=0
    exit=0

application --package-lock-only
    dependencyTotal=1
    vulnerabilityRecords=0
    exit=0

source package
    ENOLOCK
    dependencyTotal=unknown
    exit=1

source package --no-package-lock
    dependencyTotal=0
    vulnerabilityRecords=0
    exit=0

published package
    ENOLOCK
    dependencyTotal=unknown
    exit=1

published package --no-package-lock
    dependencyTotal=0
    vulnerabilityRecords=0
    exit=0

public lodash control
    dependencyTotal=1
    vulnerabilityRecords=1
    exit=1
```

The physical transformation was:

```
source package
    generator + input
        ↓ prepack
published package
    generated runtime + generated evidence
```

But the npm audit model reduced those package-local states to:

```
0 dependencies
0 vulnerability records
```

when reconstructed from their package metadata.

---

1. NPM AUDIT IS DEPENDENCY-METADATA DRIVEN

The application and `--package-lock-only` scans returned the same result even though the latter explicitly ignored `node_modules`.

For S05:

```
installed files
    did not change
vulnerability result
```

---

2. PHYSICAL PACKAGE CONTENTS ARE NOT ENOUGH FOR ORDINARY NPM AUDIT

Both the source package and published package contained meaningful executable or generated content.

Without a lockfile, normal `npm audit` failed with:

```
ENOLOCK
```

So:

```
software bytes present
    !=
npm audit inventory available
```

---

3. SOURCE AND PUBLISHED PACKAGES CAN LOOK IDENTICAL TO THE AUDIT MODEL

The source package contained the generator and input.

The published package contained generated runtime output instead.

Yet:

```
source --no-package-lock
    dependencyTotal=0

published --no-package-lock
    dependencyTotal=0
```

The audit model did not represent the transformation between them.

---

4. LIFECYCLE DECLARATIONS ARE NOT LIFECYCLE EXECUTION EVIDENCE

The published `package.json` still says:

```
"prepack": "node scripts/generate-dist.js"
```

but the actual generator is absent from the tarball.

`npm audit` exposed neither the declaration nor evidence that the lifecycle script had executed.

Therefore:

```
lifecycle declaration
    !=
lifecycle execution proof
```

---

5. GENERATED RUNTIME PROVENANCE DOES NOT BECOME VULNERABILITY IDENTITY

The published and installed package explicitly contained:

```
npm-prepack-generated
scripts/generate-dist.js
/hidden/prepack-info
```

as provenance strings.

None appeared in any audit output.

So:

```
runtime provenance
    !=
dependency identity
    !=
vulnerability visibility
```

---

6. A CLEAN AUDIT RESULT DOES NOT DESCRIBE THE COMPLETE PACKAGE-PRODUCTION HISTORY

The public lodash control proved the advisory service was working.

The absence of findings for S05 therefore means:

> npm found no known vulnerability for the dependency identities represented by the audit model.

It does not mean:

> npm inspected or reconstructed every piece of code or build-time action that produced the installed package.

T07 demonstrates:

```
package source
    !=
published package
    !=
```

```
npm dependency model
    !=
npm vulnerability result
```

and:

```
prepack execution
    can generate shipped runtime code
    while the generator and input disappear
    without that provenance becoming visible
    to npm audit
```

The practical rule is:

> Treat `npm audit` as vulnerability analysis over npm dependency metadata, not as proof of how package bytes were produced or what lifecycle code executed during publication.